C1

TravelGo: A Cloud-Powered Real-Time Travel Booking Platform Using AWS

Project Description:

TravelGo is a full-stack, cloud-based travel booking platform designed to simplify the process of reserving buses, trains, flights, and hotels through a unified interface. Built using Flask as the backend framework, the application is deployed on Amazon EC2 and leverages DynamoDB for efficient storage of user data and bookings. TravelGo allows users to register, log in, search for transportation and accommodation options, and book their travel with ease. Once a booking is confirmed or cancelled, users receive real-time email notifications powered by AWS Simple Notification Service (SNS), keeping them informed throughout their journey.

The platform's user-friendly interface supports dynamic seat selection for buses, hotel filtering based on preferences such as luxury or budget, and provides booking summaries along with centralized cancellation management. By combining cloud scalability, responsive design, and secure session handling, TravelGo delivers a seamless and real-time travel planning experience for users.

Scenario 1: Hassle-Free Multi-Mode Travel Booking Experience

TravelGo offers users a unified platform to search and book buses, trains, flights, and hotels all in one place. For instance, a user planning a trip from Hyderabad to Bangalore can log in, select their preferred mode of transport, choose from available options, and proceed to booking. Flask manages the backend operations such as retrieving travel listings and processing user input in real-time. Hosted on AWS EC2, the platform remains responsive even during high-traffic hours like weekends or holiday seasons, allowing multiple users to browse and book without delay.

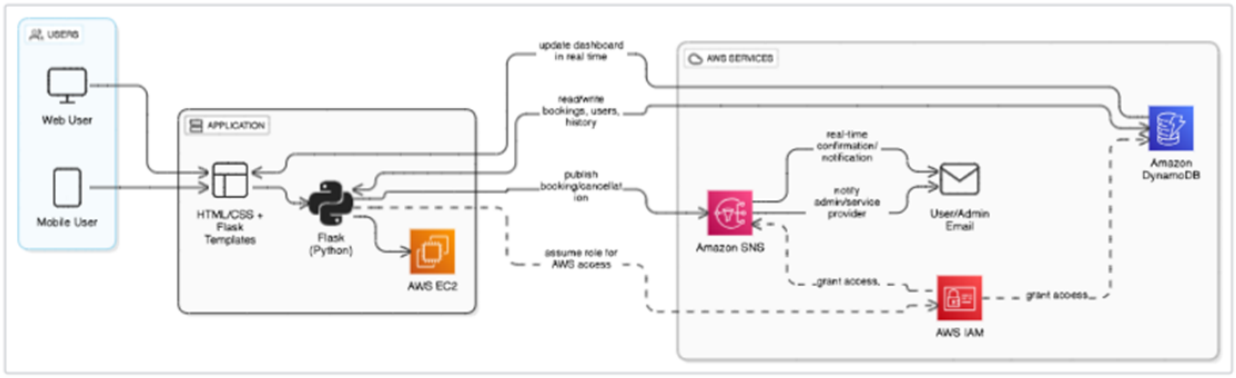
Scenario 2: Real-Time Booking Confirmation with AWS SNS

Once a booking is made—whether it's a train ticket or a hotel stay—TravelGo uses AWS SNS to instantly notify the user. For example, after a student books a hotel in Chennai, SNS sends a real-time email notification confirming the booking with all the relevant details. This notification is triggered from the Flask backend after the booking is successfully recorded in DynamoDB. Additionally, SNS can alert admin or service providers, ensuring transparency and real-time updates on every transaction.

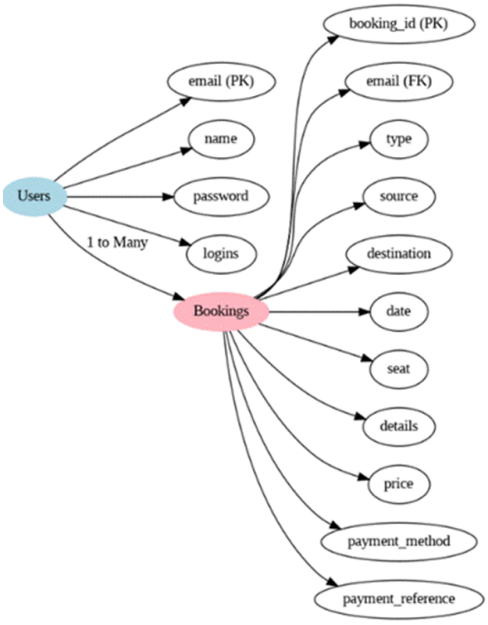
Scenario 3: Dynamic Dashboard with Personal Travel History

TravelGo features a dynamic user dashboard that displays all past and upcoming bookings for the logged-in user. For example, a user who has booked a flight and a hotel can view these bookings categorized by type, along with dates, price, and cancellation options. Flask fetches this data from AWS DynamoDB, which persistently stores all user bookings. The dashboard UI, powered by responsive HTML/CSS and Flask templates, ensures users can review or manage bookings anytime, from any device, with real-time updates and quick cancellation workflows supported.

AWS ARCHITECTURE



Entity Relationship (ER)Diagram:



Pre-requisites:

1. .**AWS Account Setup**:
   AWSAccount Setup
2. **Understanding IAM**:
   IAMOverview
3. **Amazon EC2 Basics**:
   EC2Tutorial
4. **DynamoDB Basics**:
   DynamoDB Introduction
5. **SNS Overview**:
   SNSDocumentation
6. **Git Version Control**:
   Git Documentation

Project WorkFlow:

1. **AWS Account Setup and Login**

Activity 1.1: Set up an AWS account if not already done.
Activity 1.2: Login to AWS Management console.

2. DynamoDB Database Creation and Setup

Activity 2.1: Create a DynamoDB Table.
Activity 2.2: Configure Attributes for User Data and Book Requests.

3. SNS Notification Setup

Activity 3.1: Create SNS topics for book requestnotifications.
Activity 3.2: Subscribe users and librarystaff to SNS email notifications.

4. Backend Development and Application Setup

Activity 4.1:Develop the BackendUsing Flask.
Activity 4.2: Integrate AWS Services Using boto3.

5. IAM Role Setup

Activity 5.1: Create IAM Role
Activity 5.2: Attach Policies

6. EC2 Instance Setup

Activity 6.1: Launch an EC2 instanceto host the Flask application.
Activity 6.2: Configure securitygroups for HTTP, and SSH access.

7. Deployment on EC2

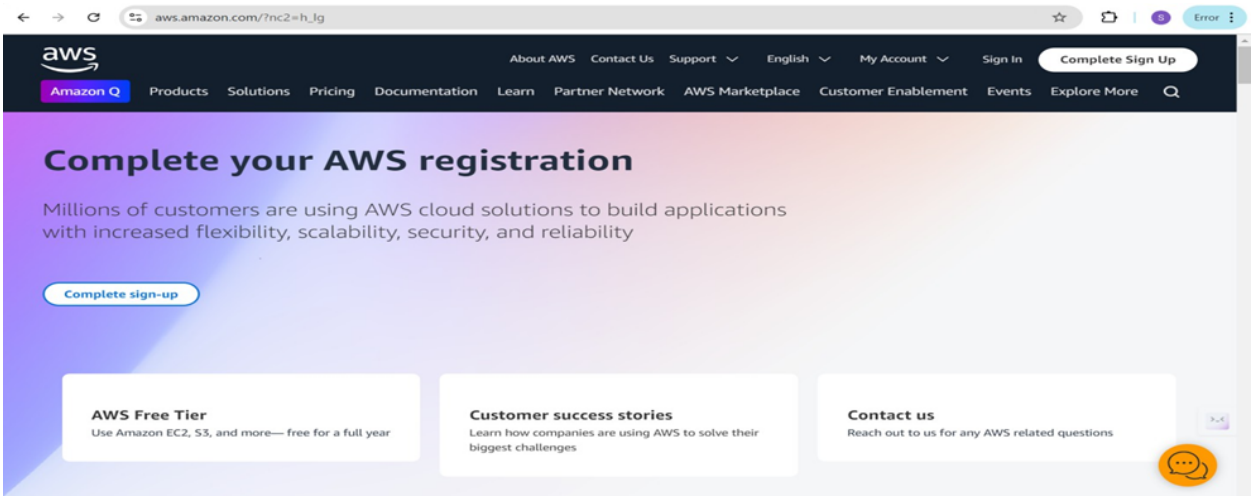
Activity 6.1: Launch an EC@ instance to host the Flask application.
Activity 6.2: configure security groups for Http, and ssh access.

8. Testing and Deployment

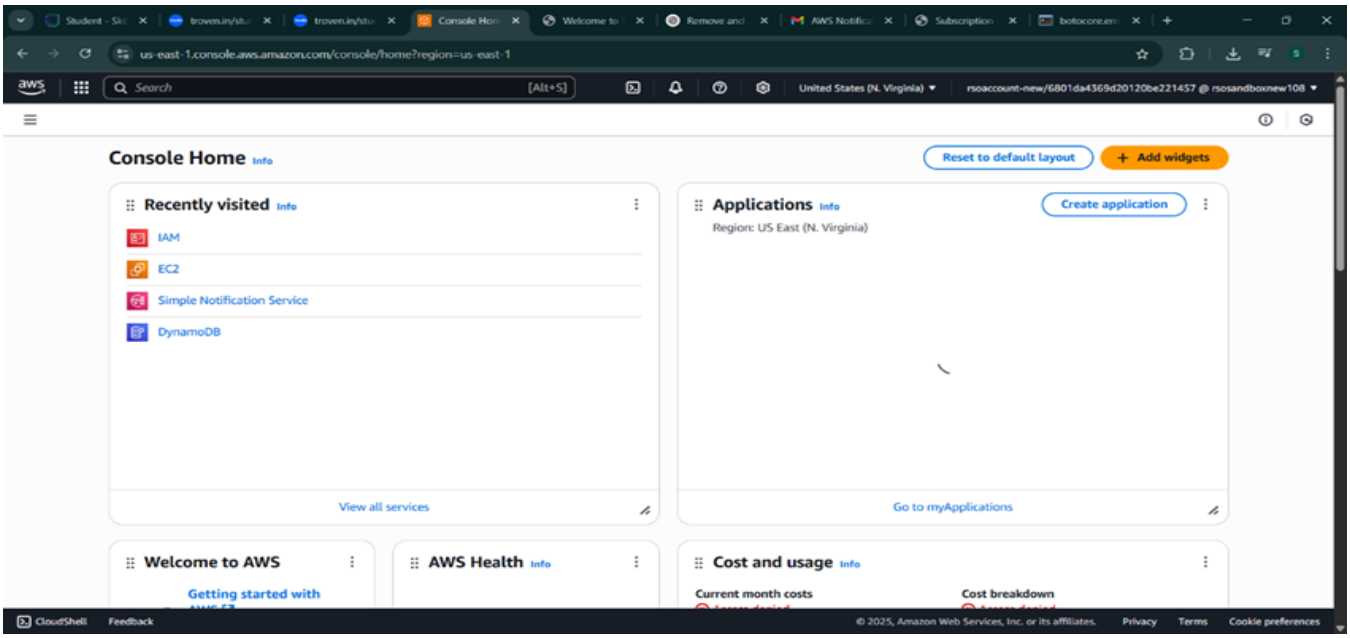
Activity 8.1: Conduct functional testing to verify user registration, login, book requests,and notifications.

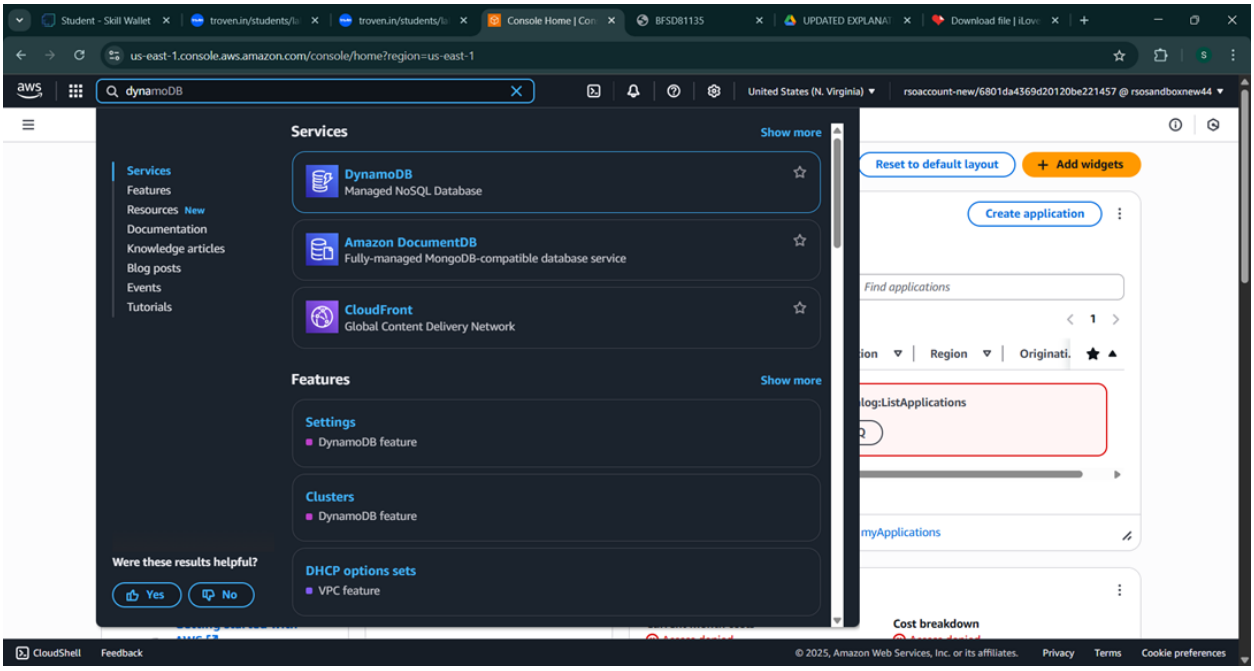
Milestone 1: AWS AccountSetup and Login

- **Activity 1.1: Set up an AWS account if not already done**
  --> Sign up for an AWS accountand configure billingsettings.



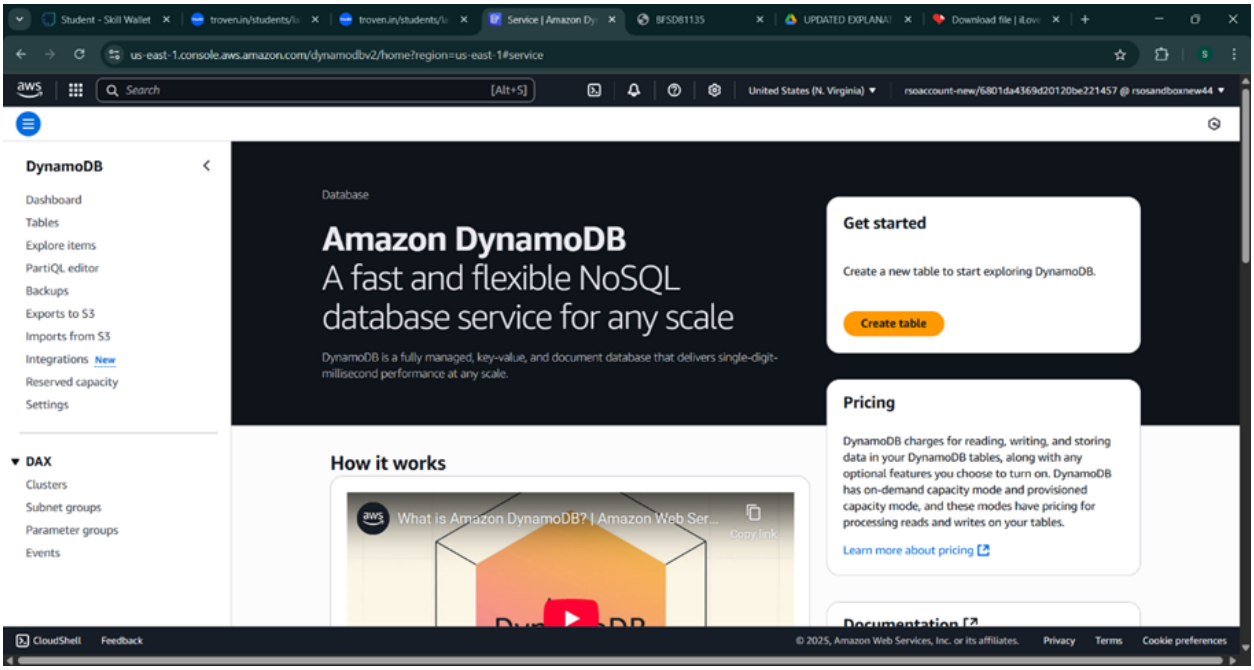
- **Activity 1.2: Log in to the AWS Management Console**
  --> After setting up your account,log in to the AWS Management Console.





Milestone 2: DynamoDBDatabase Creation and Setup

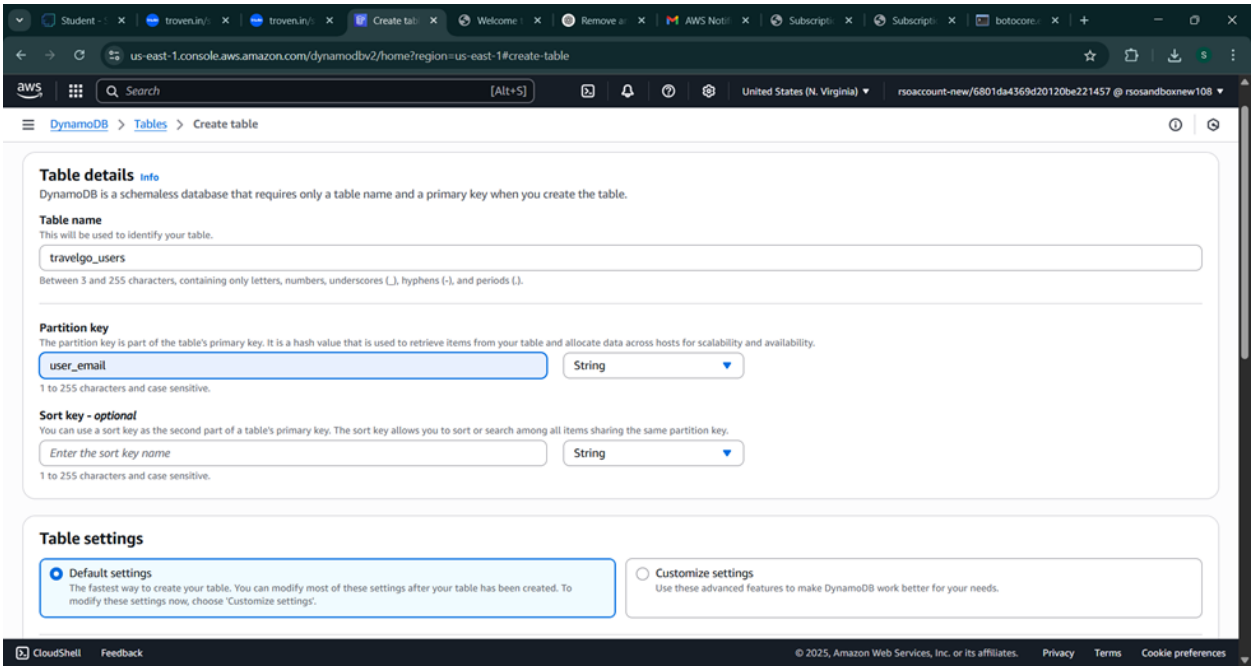
- **Activity 2.1:Navigate to the DynamoDB**
  - In the AWS Console, navigateto DynamoDB and click on create tables.



- **Activity 2.2:Create a DynamoDBtable for**

storingregistration details and book requests.

- Create Users table with partition key “user_email” with type String and click on create tables.



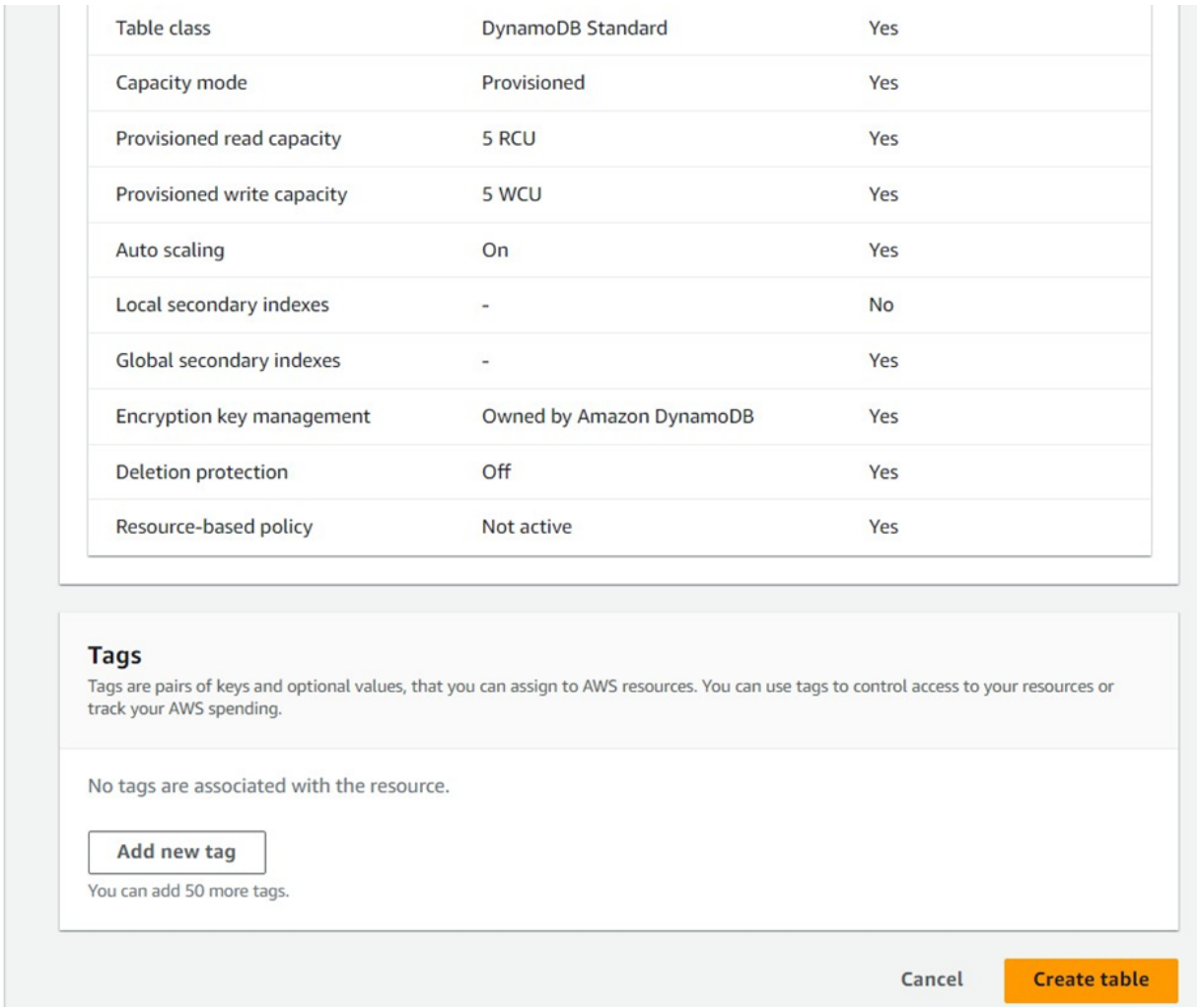
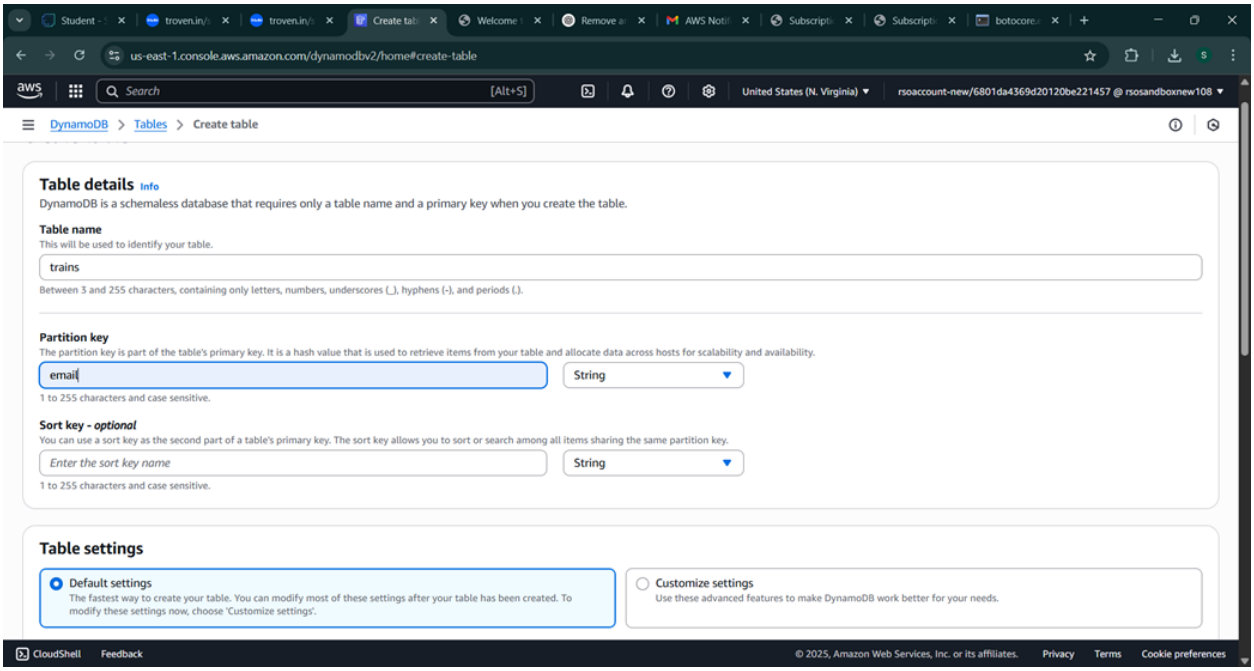
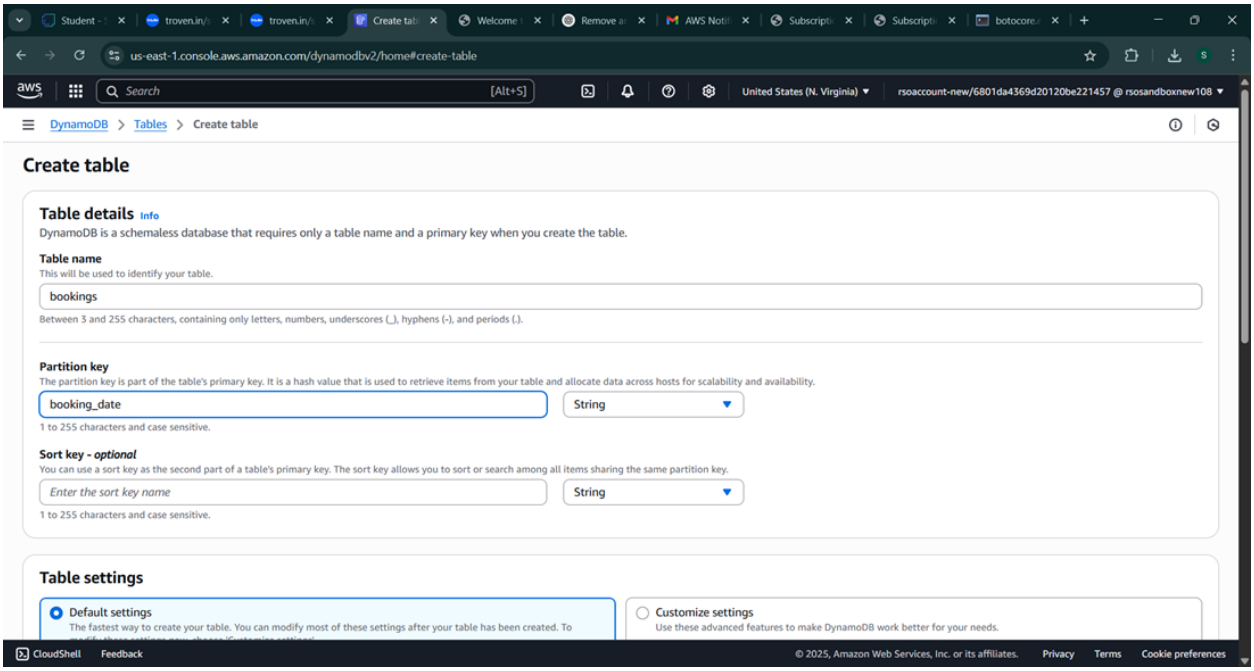
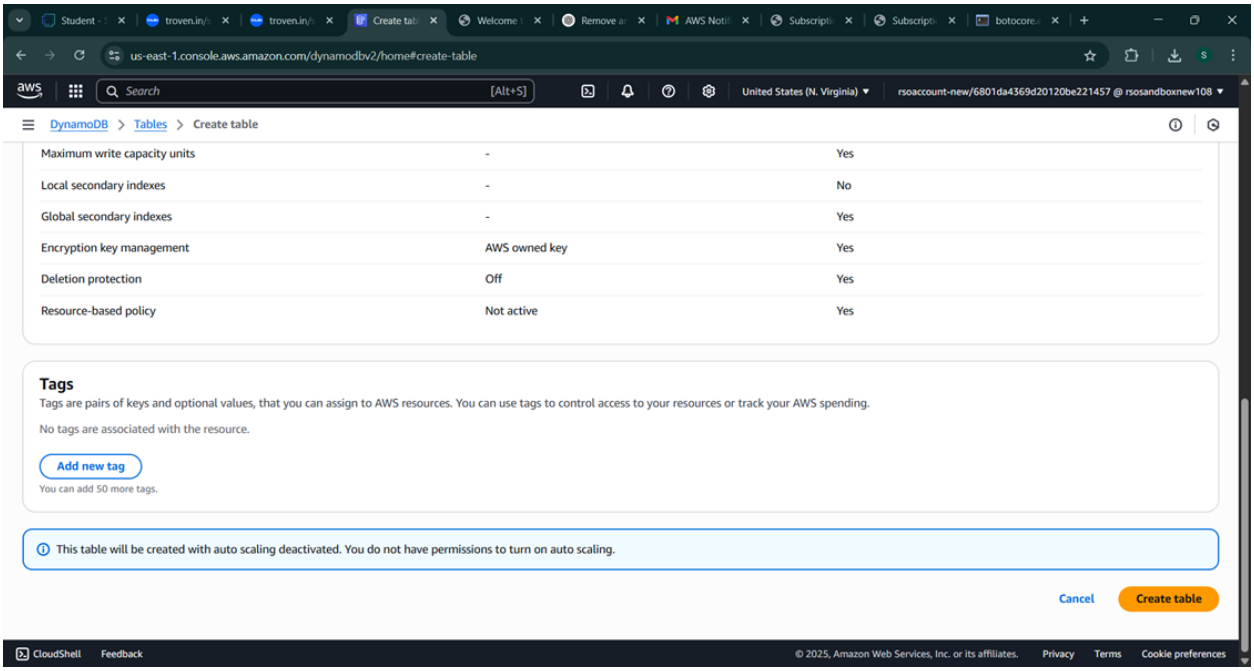


Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

- Follow the same steps to create a requests tablewith user_email as the primary key for book requests data.

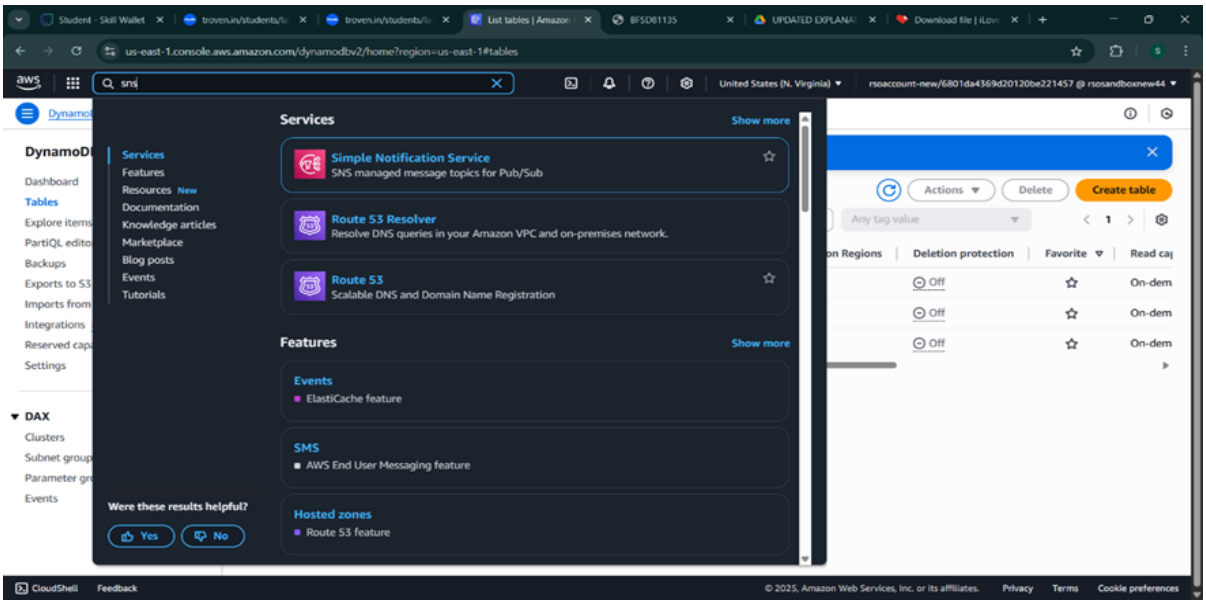

us-east-1.console.aws.amazon.com/dynamodbv2/home#create-table
DynamoDB > Tables > Create table
Table details Info
DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.
Table name
This will be used to identify your table.
trains
Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).
Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.
email
String
1 to 255 characters and case sensitive.
Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.
Enter the sort key name
String
1 to 255 characters and case sensitive.
Table settings
Default settings
The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose 'Customize settings'.
Customize settings
Use these advanced features to make DynamoDB work better for your needs.
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


us-east-1.console.aws.amazon.com/dynamodbv2/home#create-table
DynamoDB > Tables > Create table
Create table
Table details Info
DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.
Table name
This will be used to identify your table.
bookings
Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).
Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.
booking_date
String
1 to 255 characters and case sensitive.
Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.
Enter the sort key name
String
1 to 255 characters and case sensitive.
Table settings
Default settings
Customize settings
Use these advanced features to make DynamoDB work better for your needs.
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

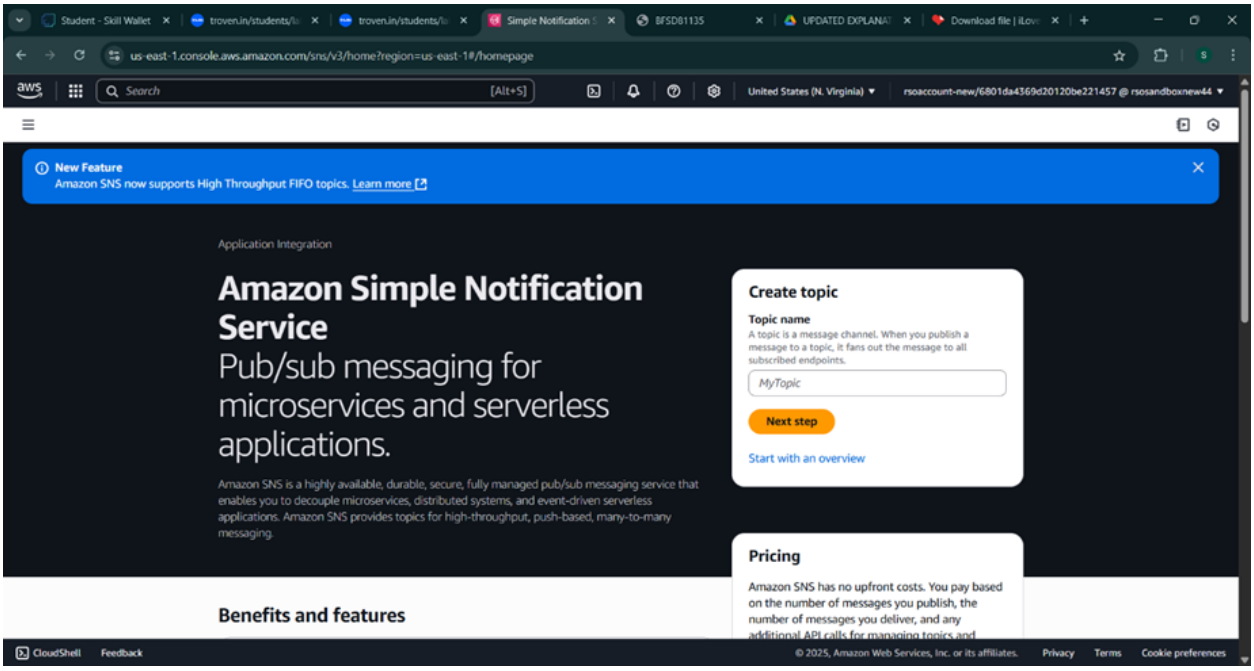


Milestone 3: SNS Notification Setup

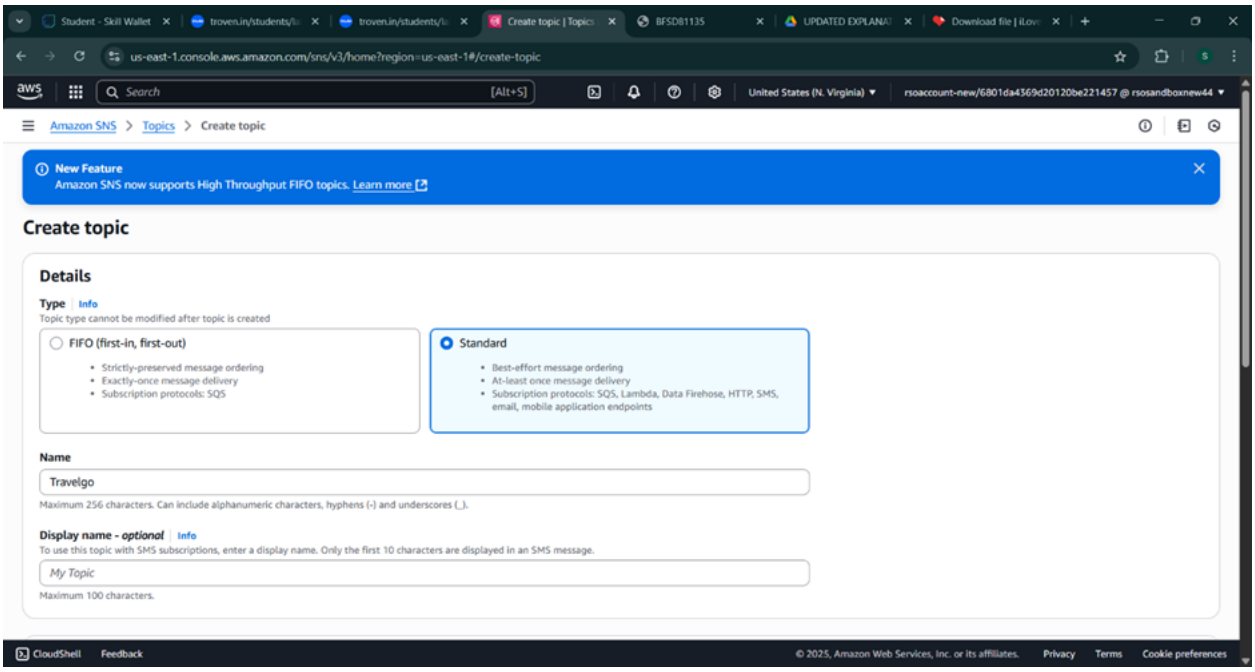
- **Activity 3.1: Create SNS topics for sending email notifications to users**



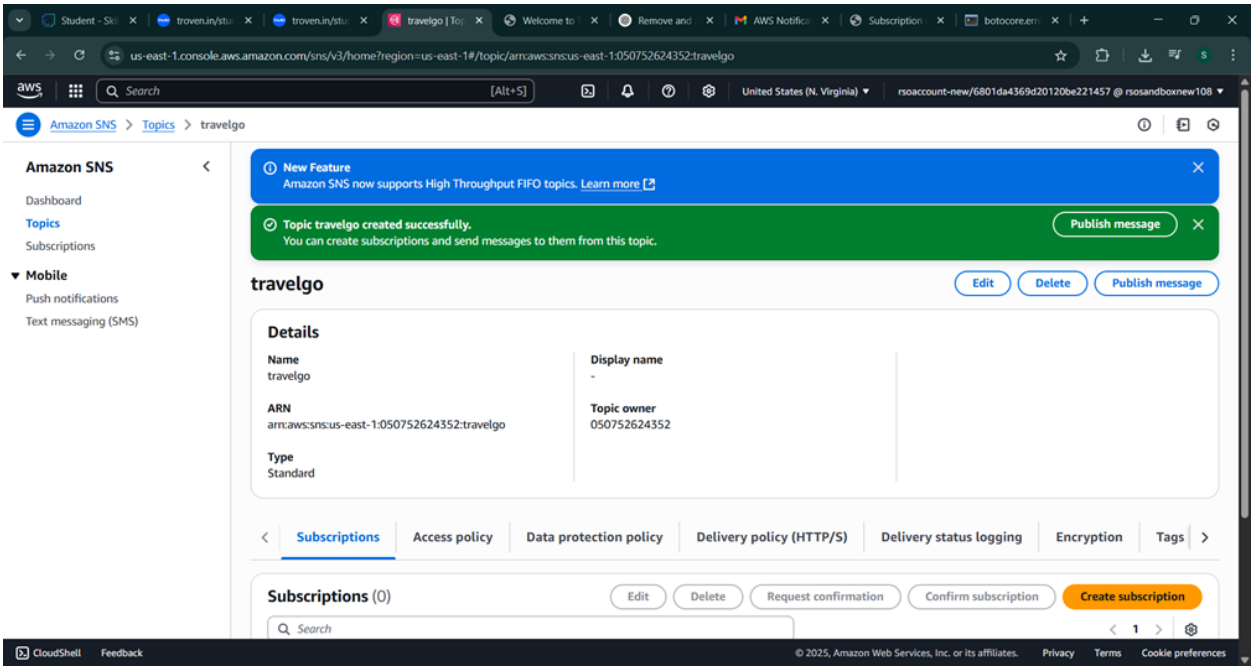
  - In the AWS Console,search for SNS and navigateto the SNS Dashboard.



- Click on **Create Topic** and choosea name for the topic.

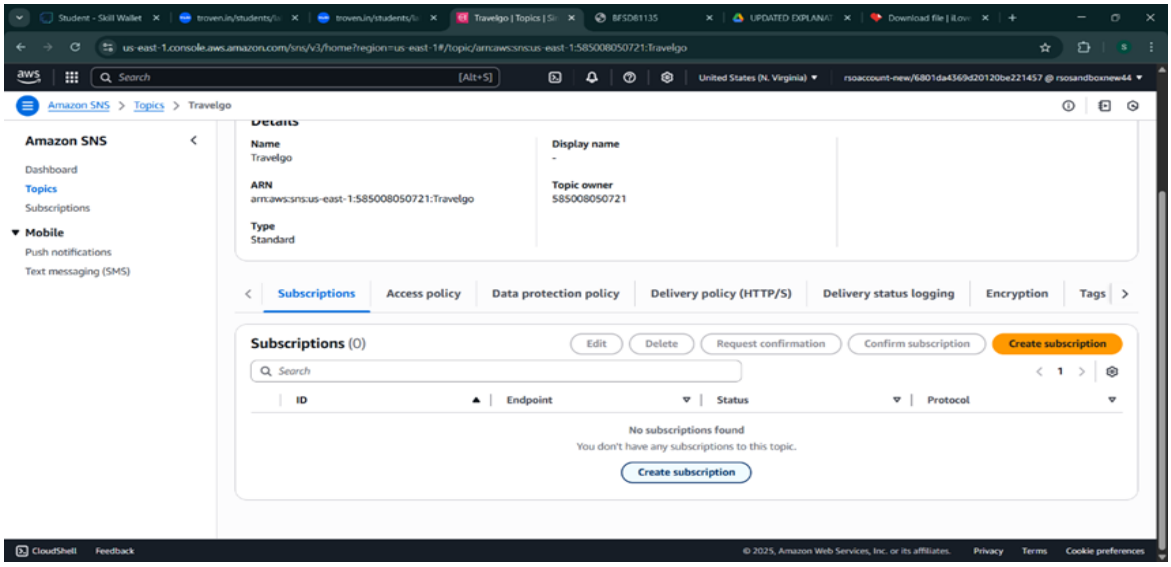


- Choose Standardtype for generalnotification use casesand Click on Create Topic.

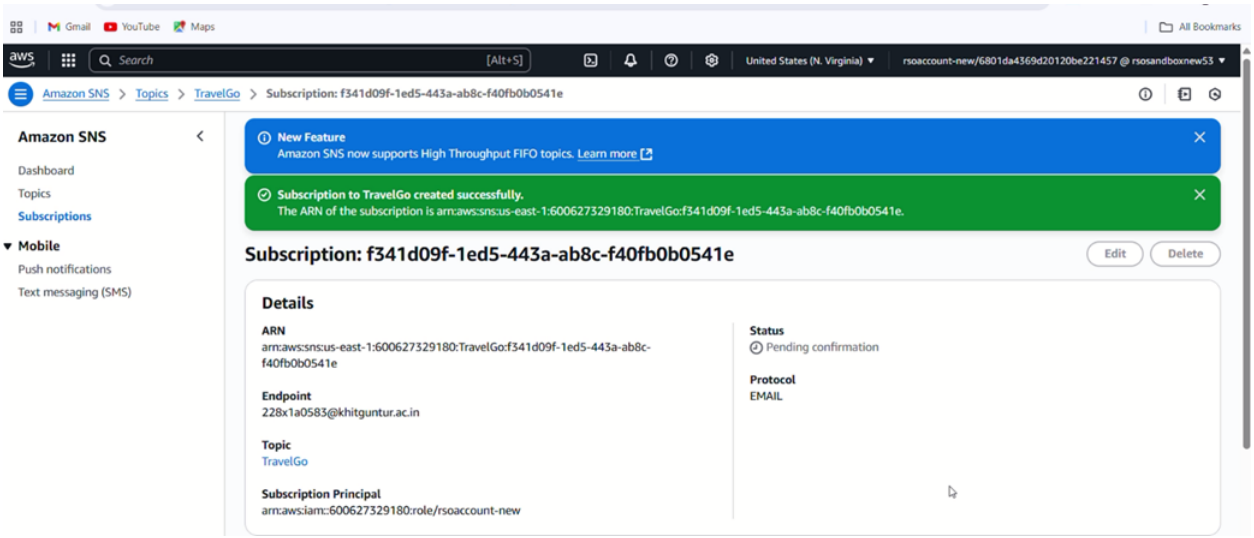


- Configure the SNS topic and note down the **Topic ARN**.

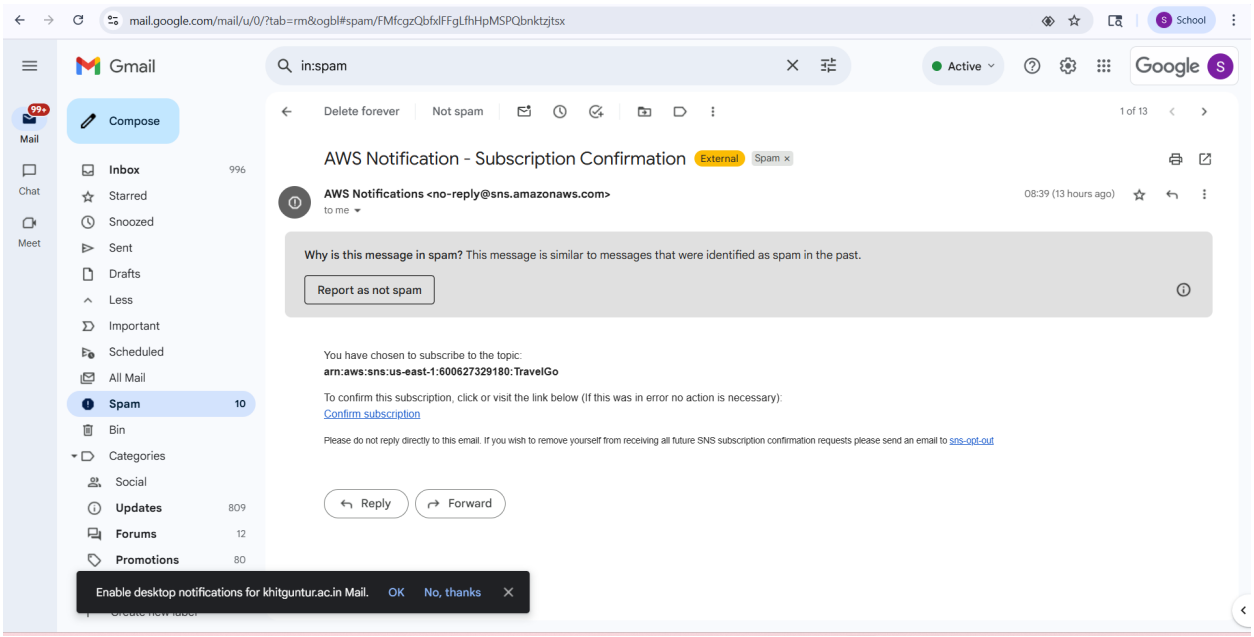
1. Activity 3.2: Subscribe users relevant SNS topics to receive real-time notifications when a book request is made.

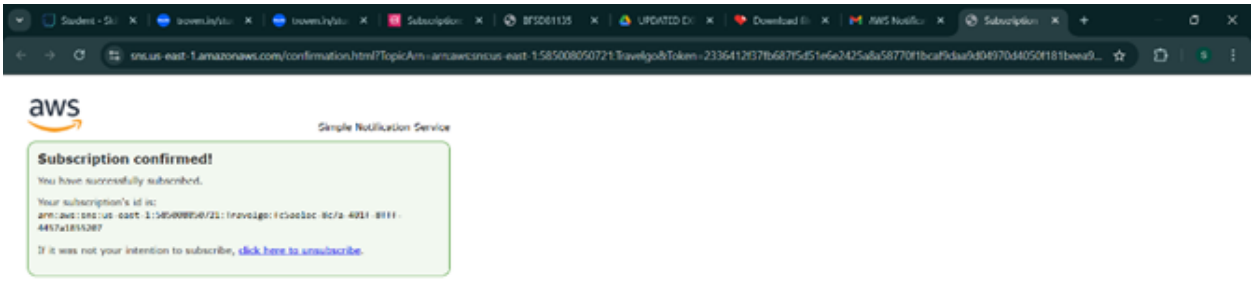


- Subscribe users to this topic via Email. When a book request is made, notifications will be sent to the subscribed emails.



- After subscription request for the mail confirmation



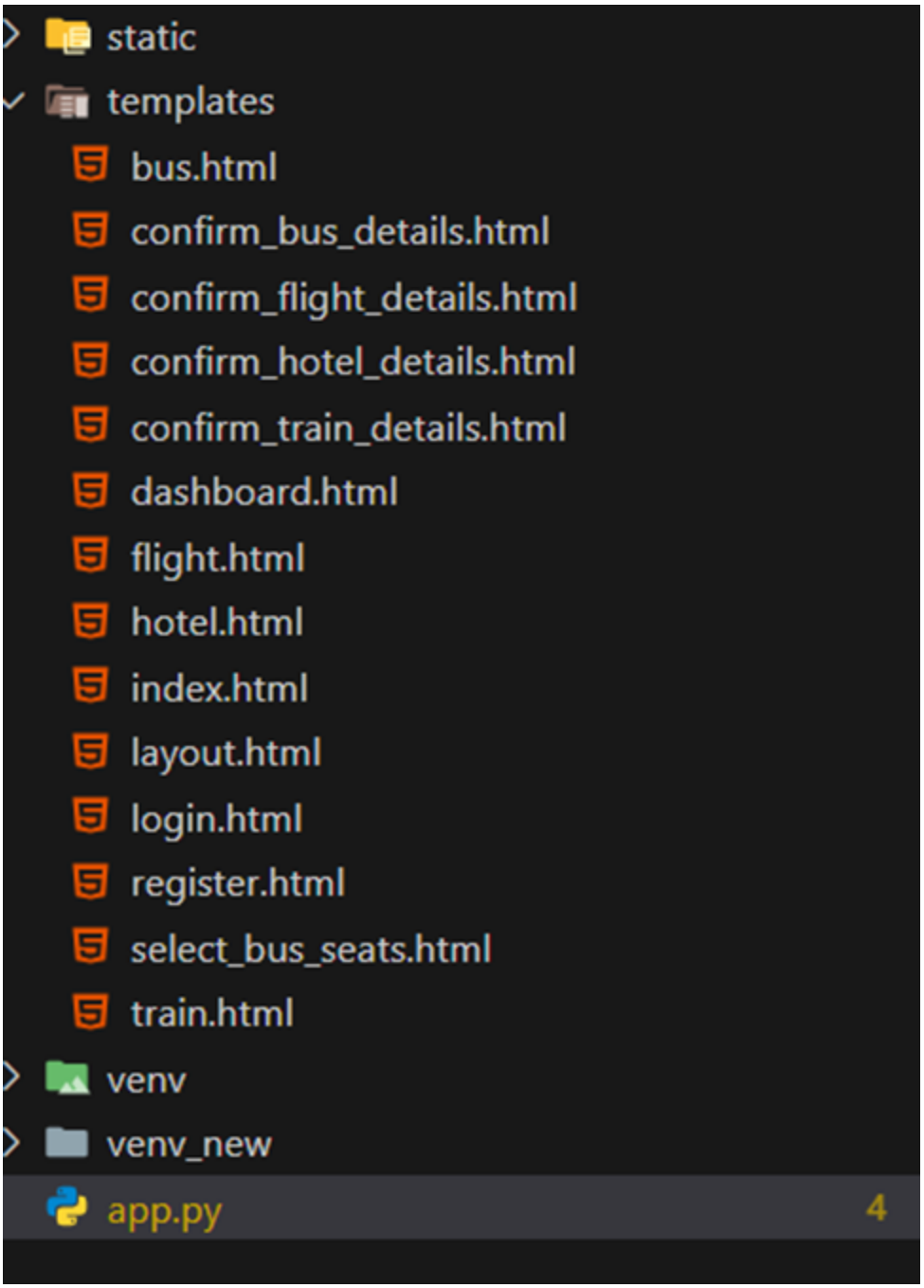


- Successfully done with the SNS mail subscription and setup, now store the ARN link.

Milestone 4:Backend Development and Application Setup

- **Activity 4.1: Develop the backend using Flask**

- File Explorer Structure



Description: set up the **TravelGO** project with an app.py file, a static/ folderfor assets, and a templates/ directory containing all required HTML pages like home, login, register, booking-specific pages (e.g., bus.html, train.html

Description of the code :

- **Flask App Initialization**

```
from flask import Flask, render_template, request, redirect, url_for, session, jsonify, flash
import boto3
from boto3.dynamodb.conditions import Key, Attr
from werkzeug.security import generate_password_hash, check_password_hash
from datetime import datetime
from decimal import Decimal
import uuid
import random
```

Description: import essential librariesincluding Flask utilitiesfor routing, Boto3 for DynamoDB operations,

```
app = Flask(__name__)
```

Description: initialize the Flask application instanceusing Flask(_name) to start building the web app.

- **Dynamodb Setup:**

```
# AWS Setup using IAM Role
REGION = 'us-east-1'  # Replace with your actual AWS region
dynamodb = boto3.resource('dynamodb', region_name=REGION)
sns_client = boto3.client('sns', region_name=REGION)

users_table = dynamodb.Table('travelgo_users')
trains_table = dynamodb.Table('trains') # Note: This table is declared bu
bookings_table = dynamodb.Table('bookings')
```

Description: initialize the DynamoDBresource for the us-east-1 region and set up accessto the Users and Requests tables for storing user details and book requests.

(OR)

- **MongoDB Setup:**

```
# MongoDB connection
client = MongoClient('mongodb+srv://Mohan:er2af5Q1UBCewcmZ@cluster0.di6bzxd.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0')
db = client['travel_booking_db']
# Collections
users_collection = db['users']
trains_collection = db['trains']
bookings_collection = db['bookings']
```

- **SNS Connection**

```
SNS_TOPIC_ARN = 'arn:aws:sns:us-east-1:418272775181:TravelGo:b7d6f5bc-7710-49de-97bc-ddecff5fd023'
topic ARN

# Function to send SNS notifications

def send_sns_notification(subject, message):
    try:
        sns_client.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject=subject,
            Message=message
        )
    except Exception as e:
        print(f"SNS Error: Could not send notification - {e}")
        # Optionally, flash an error message to the user or log it more robustly.
```

Description: Configure **SNS** to send notifications when a book request is submitted. Paste your stored ARN link in the sns_topic_arn space, along with the region_name where the SNS topic is created.

- **Routes for Web Pages**

  - **Home Route:**

```
# Home route redirects to Registration page
@app.route('/')
def home():
    return redirect(url_for('register'))
```

Description: define the home route / to automatically redirect users to the register page when they access the base URL.

- **Register Route:**

```python
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        # existing = users_collection.find_one({'email': email})  # MongoDB
        existing = users_table.get_item(Key={'email': email})  # DynamoDB
        if 'Item' in existing:
            flash('Email already exists!', 'error')
            return render_template('register.html')
        hashed_password = generate_password_hash(password)
        # users_collection.insert_one({'email': email, 'password': hashed_password})  # MongoDB
        users_table.put_item(Item={'email': email, 'password': hashed_password})  # DynamoDB
        flash('Registration successful! Please log in.', 'success')
        return redirect(url_for('login'))
    return render_template('register.html')
```

Description: define /register route to validate registration form fields, hash the user password using Bcrypt, store the new user in DynamoDB with a login count, and send an SNS notification on successful registration.

- **login Route (GET/POST)**:

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    if 'username' in session:
        session.pop('username', None)
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        # user = users_collection.find_one({'email': email})  # MongoDB
        user = users_table.get_item(Key={'email': email})  # DynamoDB
        if 'Item' in user and check_password_hash(user['Item']['password'], password):
            session['email'] = email
            flash('Logged in successfully!', 'success')
            return redirect(url_for('dashboard'))
        else:
            flash('Invalid email or password!', 'error')
            return render_template('login.html')
    return render_template('login.html')
```

Description: define /login route to validate user credentials against DynamoDB, check the password usingBcrypt, update the login counton successful authentication, and redirect users to the home page

- **Home,Bus,Train,Flight,Hotel Routes:**

```python
@app.route('/dashboard')
def dashboard():
    if 'email' not in session:
        return redirect(url_for('login'))

    user_email = session['email']
    # user_bookings = list(bookings_collection.find({'user_email': user_email}).sort('booking_date', -1))  # MongoDB
    response = bookings_table.query(
        KeyConditionExpression=Key('user_email').eq(user_email),
        ScanIndexForward=False
    )
    bookings = response.get('Items', [])
    for booking in bookings:
        if 'total_price' in booking:
            try:
                booking['total_price'] = float(booking['total_price'])
            except Exception:
                booking['total_price'] = 0.0

    for booking in bookings:
        if '_id' in booking and isinstance(booking['_id'], ObjectId):
            booking['_id'] = str(booking['_id'])

        # Determine vehicle_type for display based on booking_type
        if booking.get('booking_type') == 'bus':
            booking['vehicle_type'] = booking.get('type', 'Bus')
            # Add seat information for bus bookings
            if booking.get('selected_seats'):
                booking['seats_display'] = ', '.join(booking['selected_seats'])
            else:
                booking['seats_display'] = 'N/A'
        elif booking.get('booking_type') == 'train':
            booking['vehicle_type'] = f"Train {booking.get('train_number', '')}" if booking.get('train_number') else "Train"
            # Add seat information for train bookings
            if booking.get('selected_seats'):
                booking['seats_display'] = ', '.join(booking['selected_seats'])
            else:
                booking['seats_display'] = 'N/A'
        elif booking.get('booking_type') == 'flight':
            booking['vehicle_type'] = f"Flight {booking.get('flight_number', '')} ({booking.get('airline', '')})"
            # Add seat information for flights
            if booking.get('selected_seats'):
                booking['seats_display'] = ', '.join(booking['selected_seats'])
            else:
                booking['seats_display'] = 'N/A'
        else:
            booking['vehicle_type'] = booking.get('booking_type', 'N/A')

    return render_template('dashboard.html', username=user_email, bookings=bookings)
```


Description: define /dashboard-page to render the main homepage,to handle booking selection and redirection.

1. **confirmbooking Routes:**

```python
@app.route('/train')
def train():
    if 'email' not in session:
        return redirect(url_for('login'))
    return render_template('train.html')

@app.route('/confirm_train_details')
def confirm_train_details():
    if 'email' not in session:
        return redirect(url_for('login'))

    name = request.args.get('name')
    train_number = request.args.get('trainNumber')
    source = request.args.get('source')
    destination = request.args.get('destination')
    departure_time = request.args.get('departureTime')
    arrival_time = request.args.get('arrivalTime')
    price_per_person = float(request.args.get('price'))
    travel_date = request.args.get('date')
    num_persons = int(request.args.get('persons'))
    train_id = request.args.get('trainId')

    total_price = price_per_person * num_persons

    booking_details = {
        'name': name,
        'train_number': train_number,
        'source': source,
        'destination': destination,
        'departure_time': departure_time,
        'arrival_time': arrival_time,
        'price_per_person': Decimal(price_per_person),
        'travel_date': travel_date,
        'num_persons': num_persons,
        'total_price': Decimal(total_price),
        'item_id': train_id,
        'booking_type': 'train',
        'user_email': session['email']
    }
    session['pending_booking'] = booking_details
    return render_template('confirm_train_details.html', booking=booking_details)
```

Description: define /request-form route to capture book request details from users, store the request in DynamoDB, send a thank-you email to the user, notify the admin,

and confirmsubmission with a success message.

Exit Route:

```
@app.route('/logout')
def logout():
    session.pop('email', None)
    flash('You have been logged out.', 'info')
    return redirect(url_for('index'))
```

Description: define /logout routethe index.html page to renderwhen the user chooses to leave or close the application.

Deployment Code:

```
if __name__ == '__main__':

    app.run(debug=True, host='0.0.0.0')
```

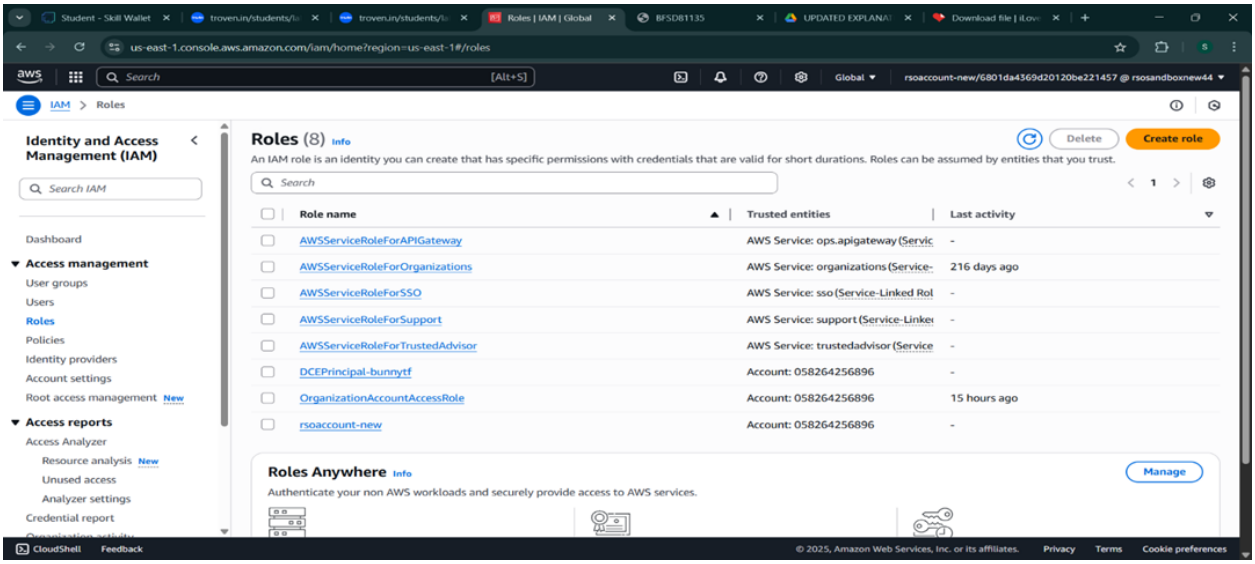
Description: start the Flask serverto listen on all networkinterfaces (`0.0.0.0`) with debug mode enabled for development and testing.

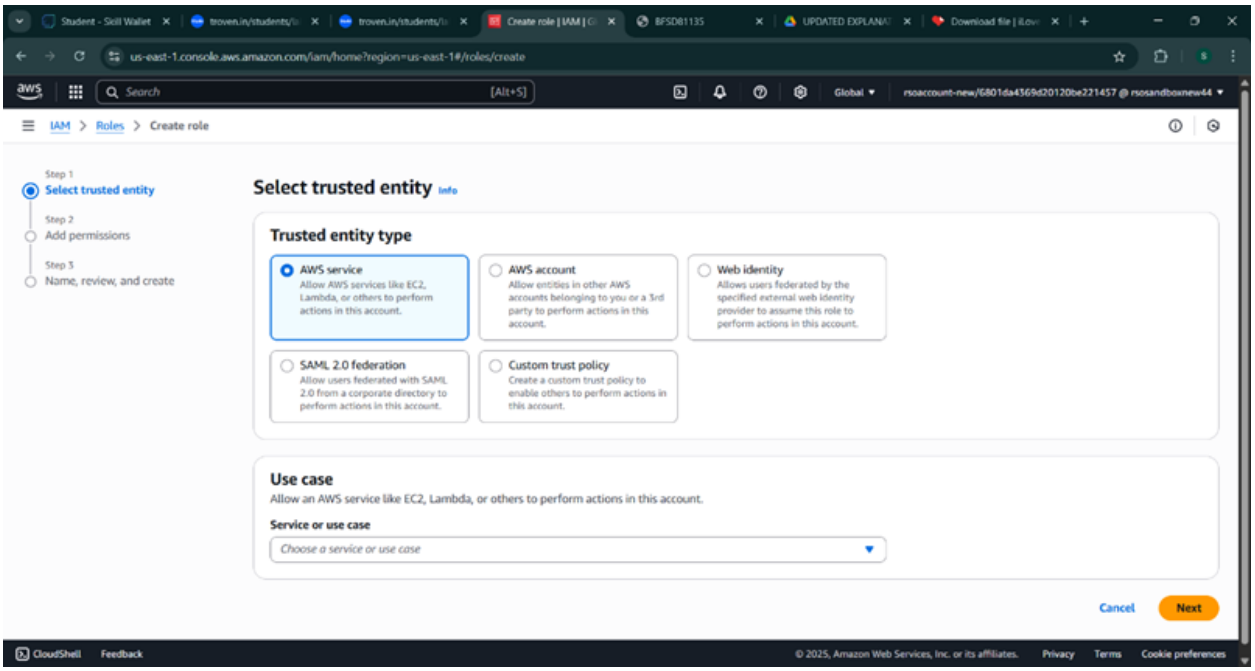
Milestone 5: IAM Role Setup

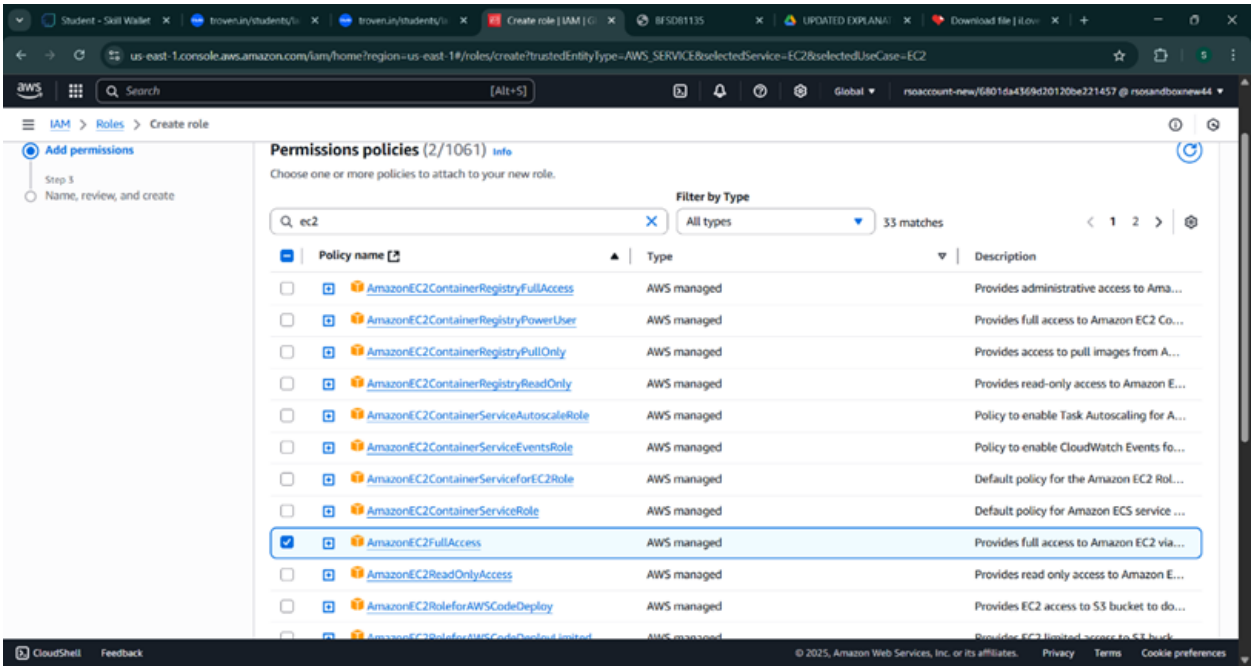
- **Activity 5.1:Create IAM Role.**
    - In the AWS Console,go to IAM and createa new IAM Role for EC2 to interact with DynamoDB and SNS.

SMARTBRIDGE
Let's Bridge the Gap

Smart Internz



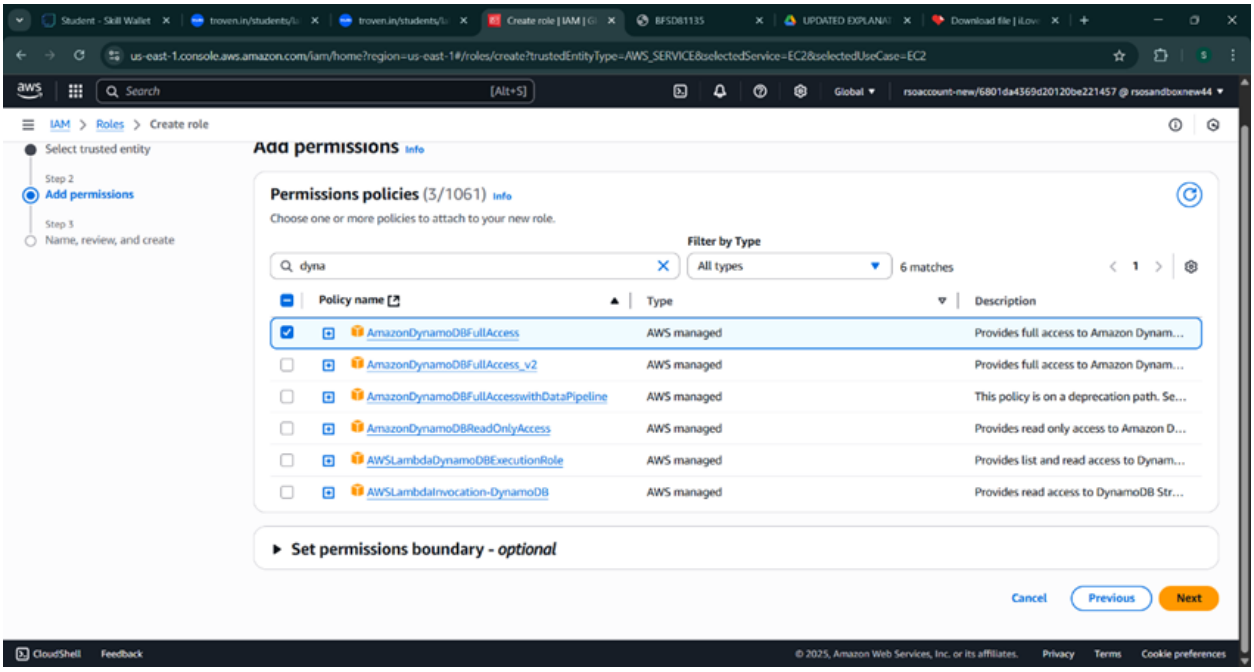


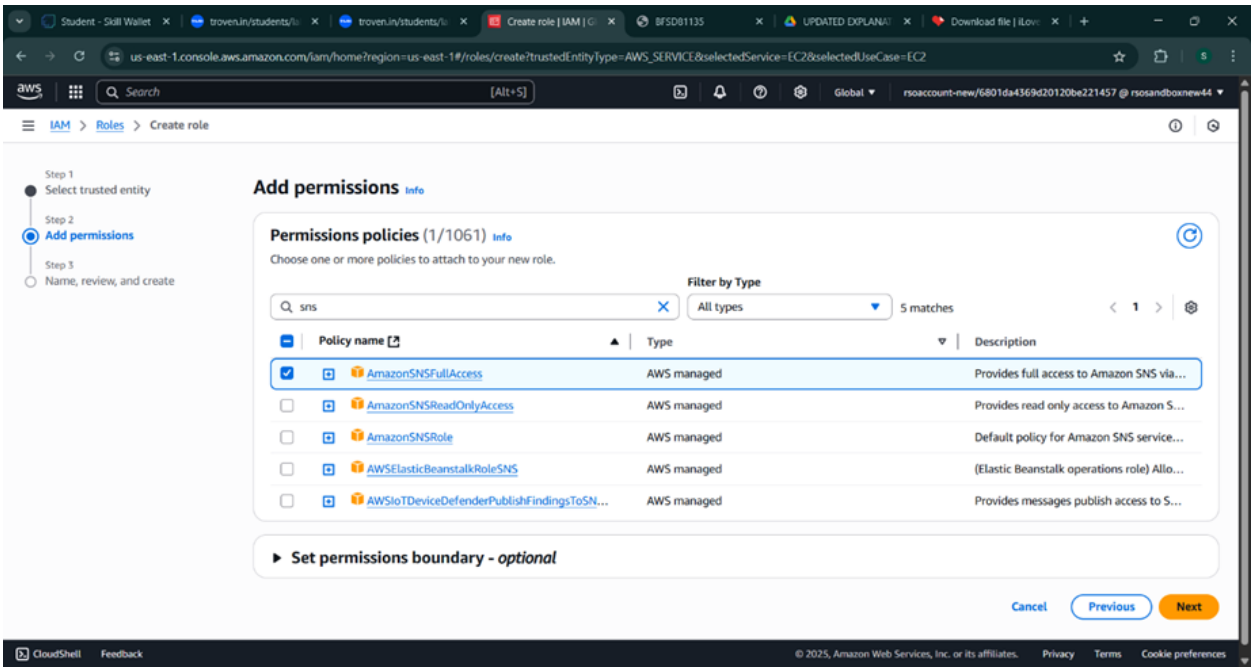


- **Activity 5.2: Attach Policies.**

Attach the following policiesto the role:

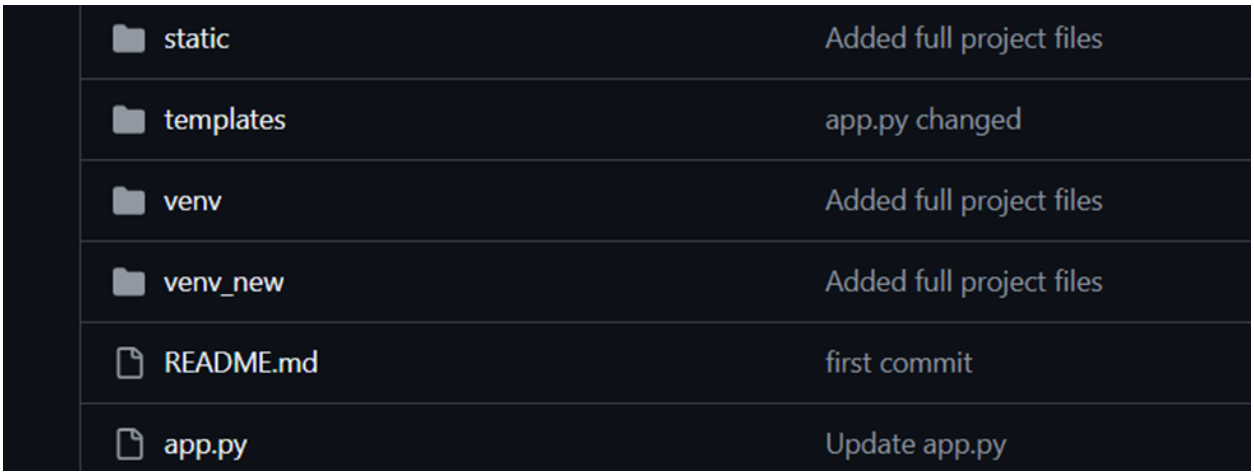
- AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.
- AmazonSNSFullAccess: Grants EC2 the ability to send notifications via SNS.
- AmazonEC2FullAccess: Allows the user to create a EC2 instance.



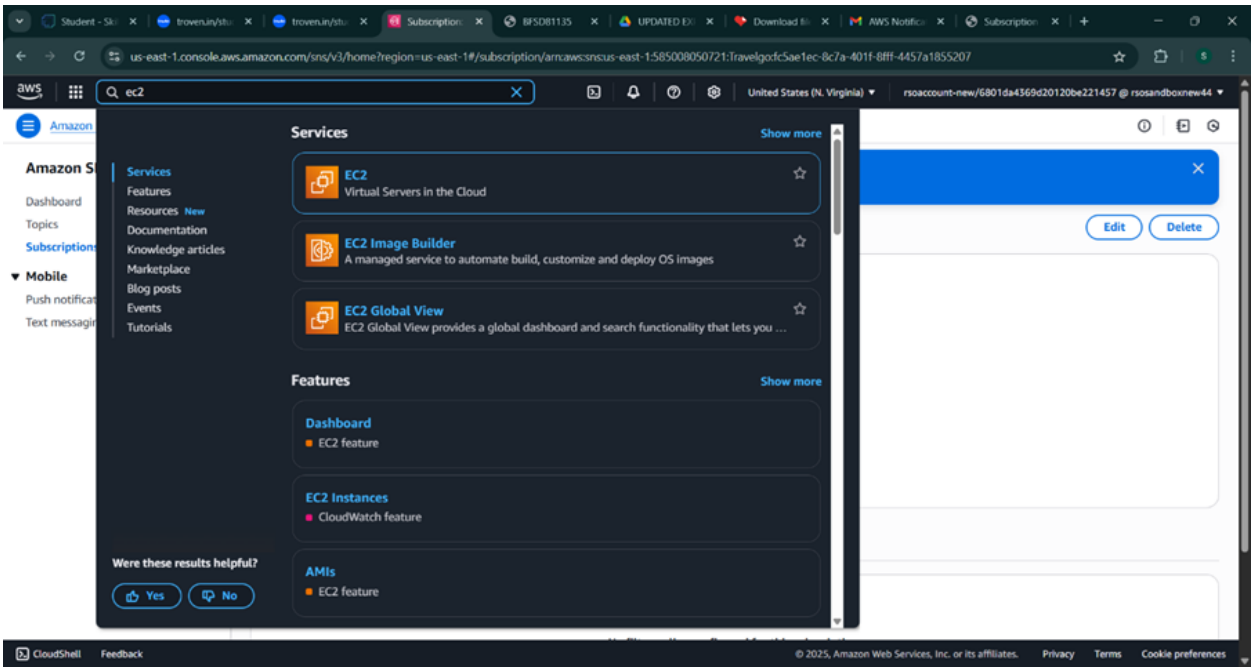


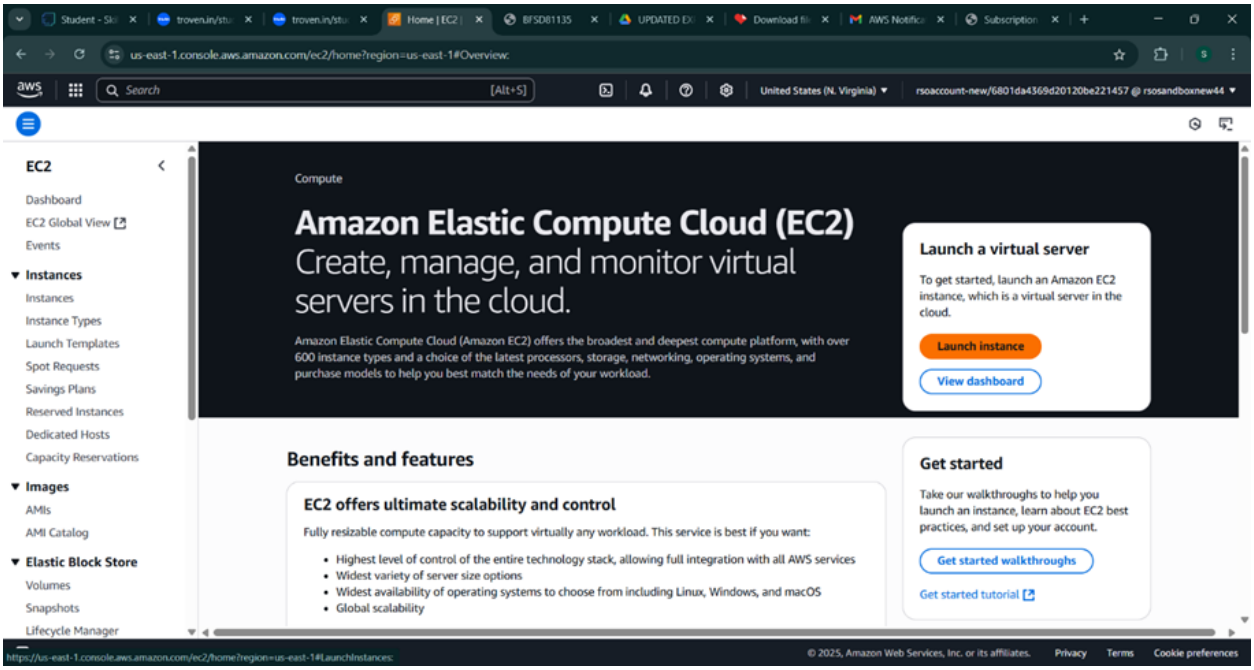
Milestone 6: EC2 InstanceSetup

- **Note: Load your Flask app and Html files into GitHubrepository.**

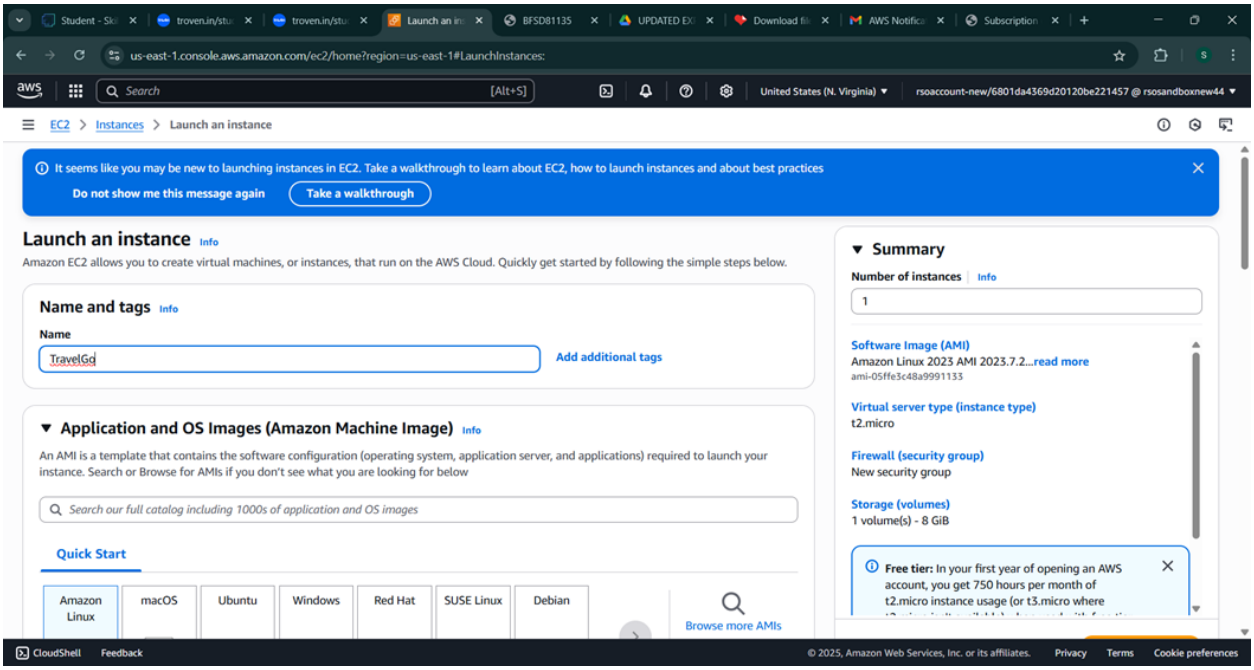


- **Activity 6.1: Launch an EC2 instance to host the Flask application.**
  - **Launch EC2 Instance**
    - In the AWS Console,navigate to EC2 and launcha new instance.

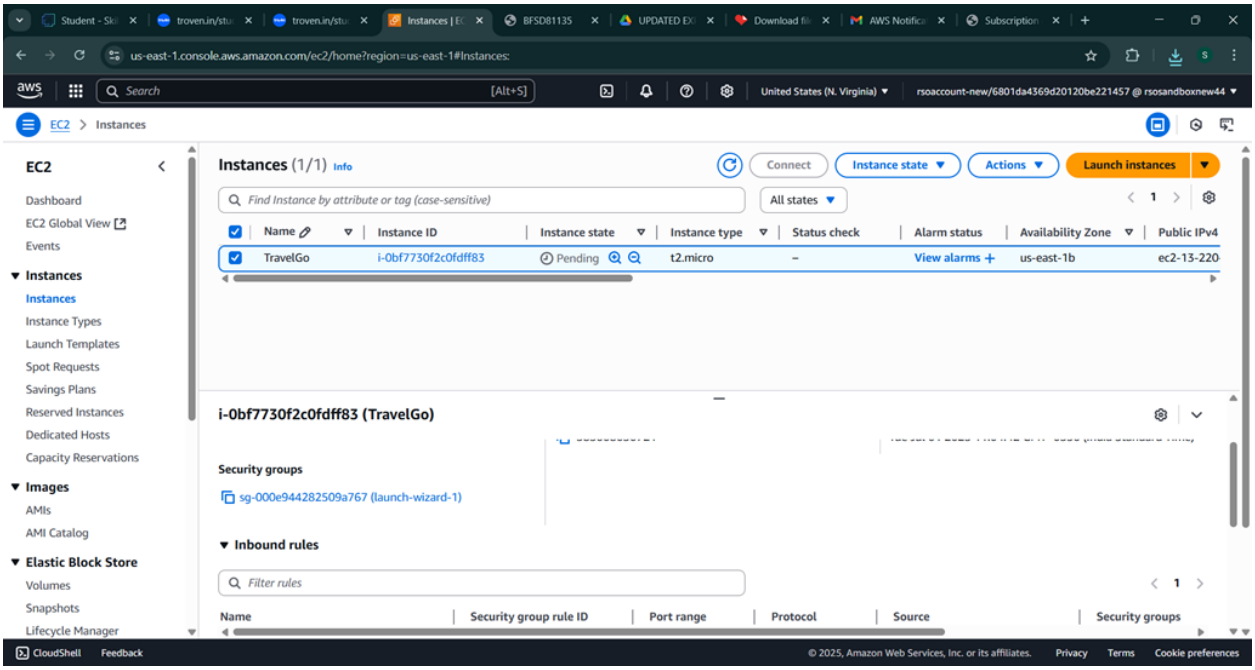




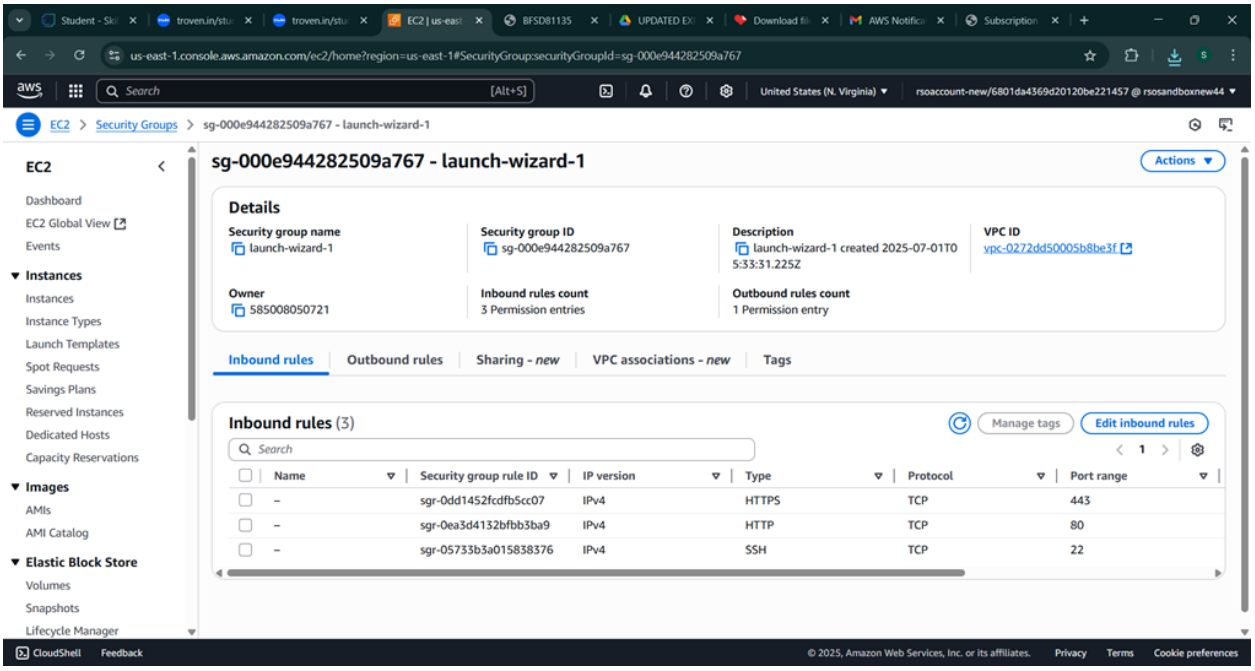
- Click on Launchinstance to launchEC2 instance
- Choose Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instancetype (free-tier eligible).



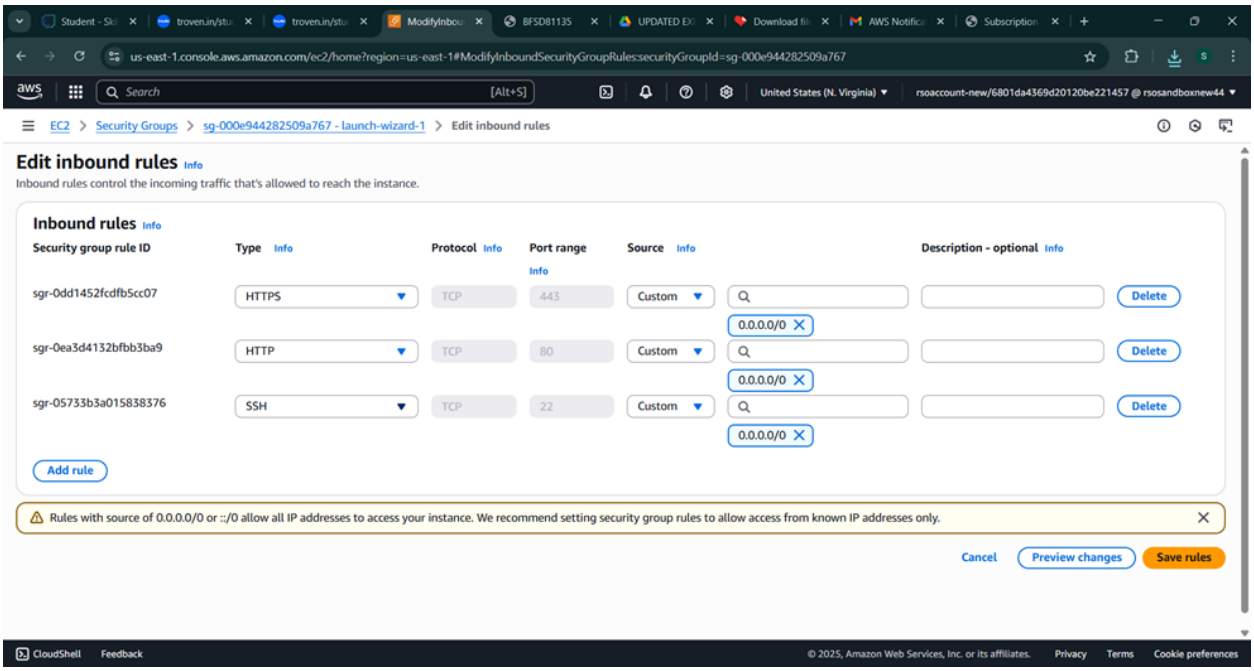
- Create and downloadthe key pair for Serveraccess.

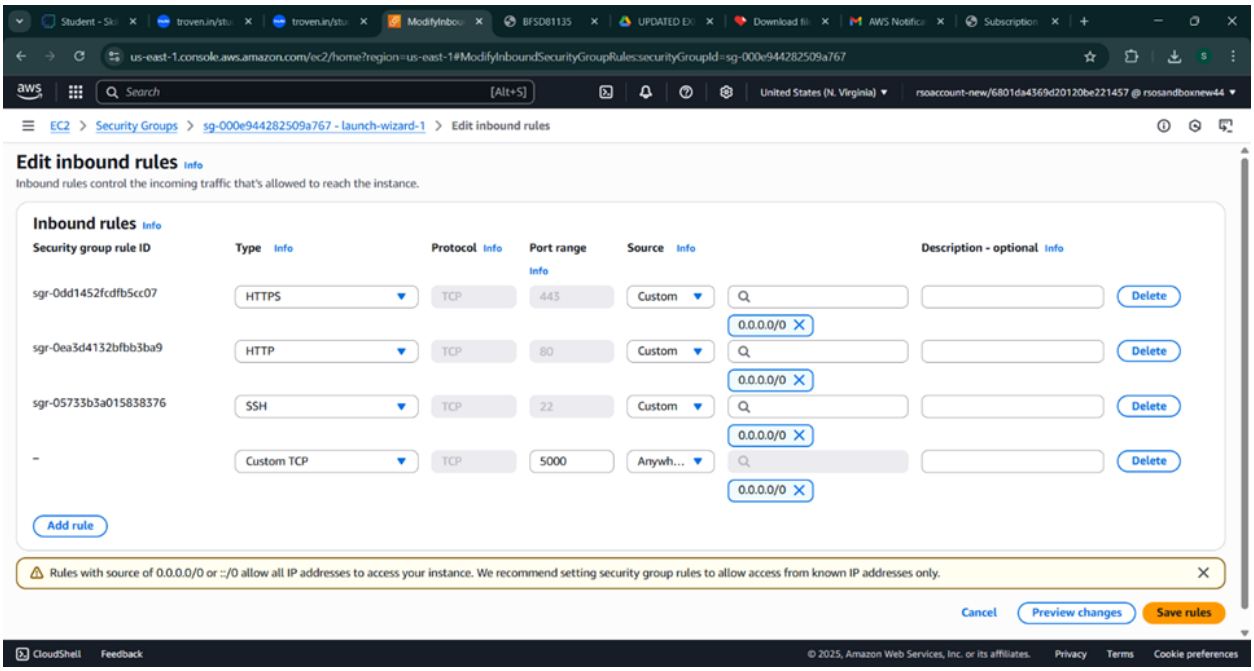


- **Activity 6.2:Configure securitygroups for HTTP,and SSH access.**

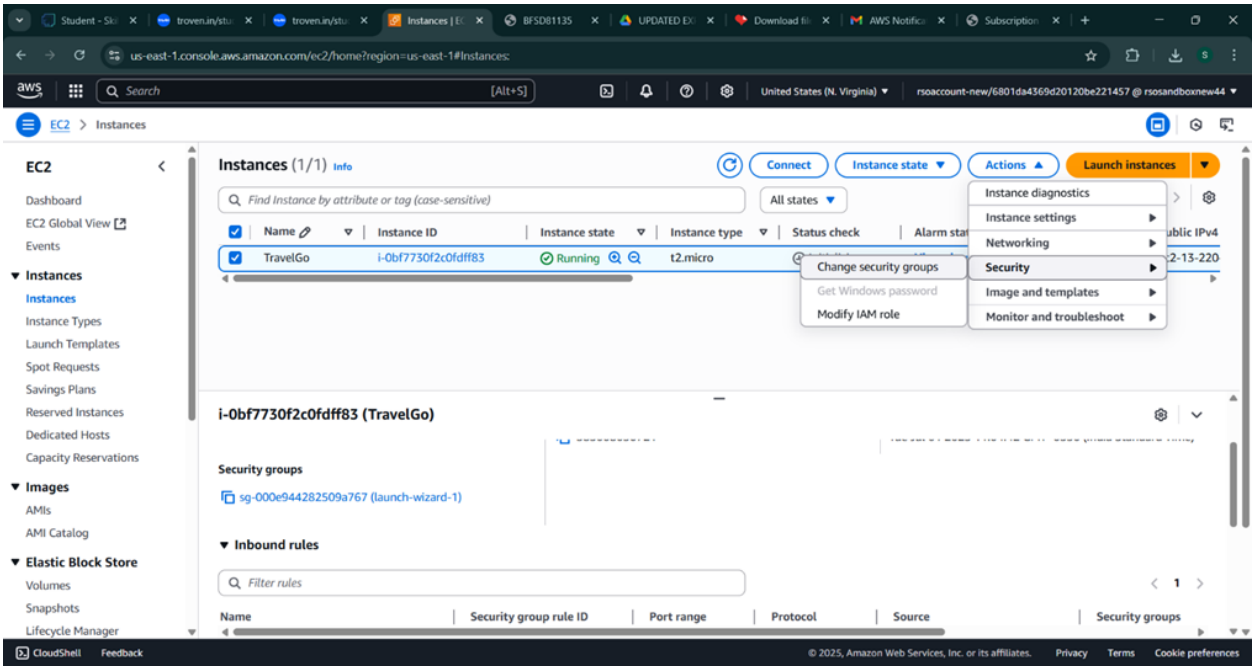


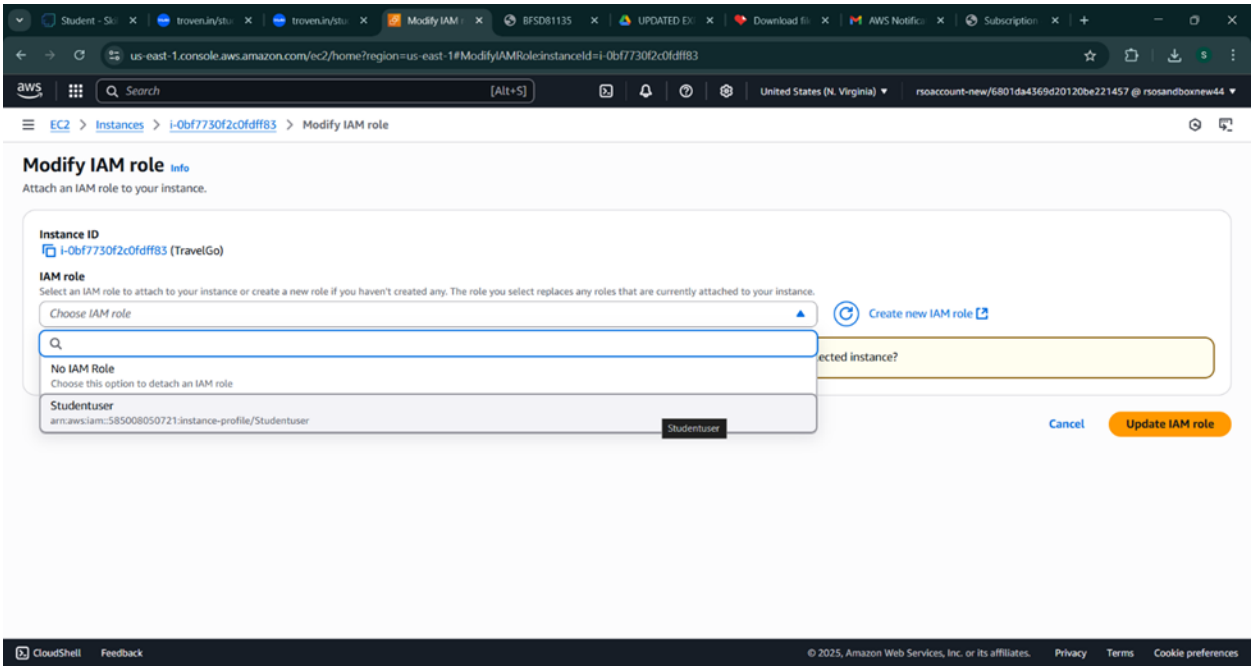
- **Activity 6.3: Add a custom inbound rule with port range 5000 and source Anywhere IPv4**



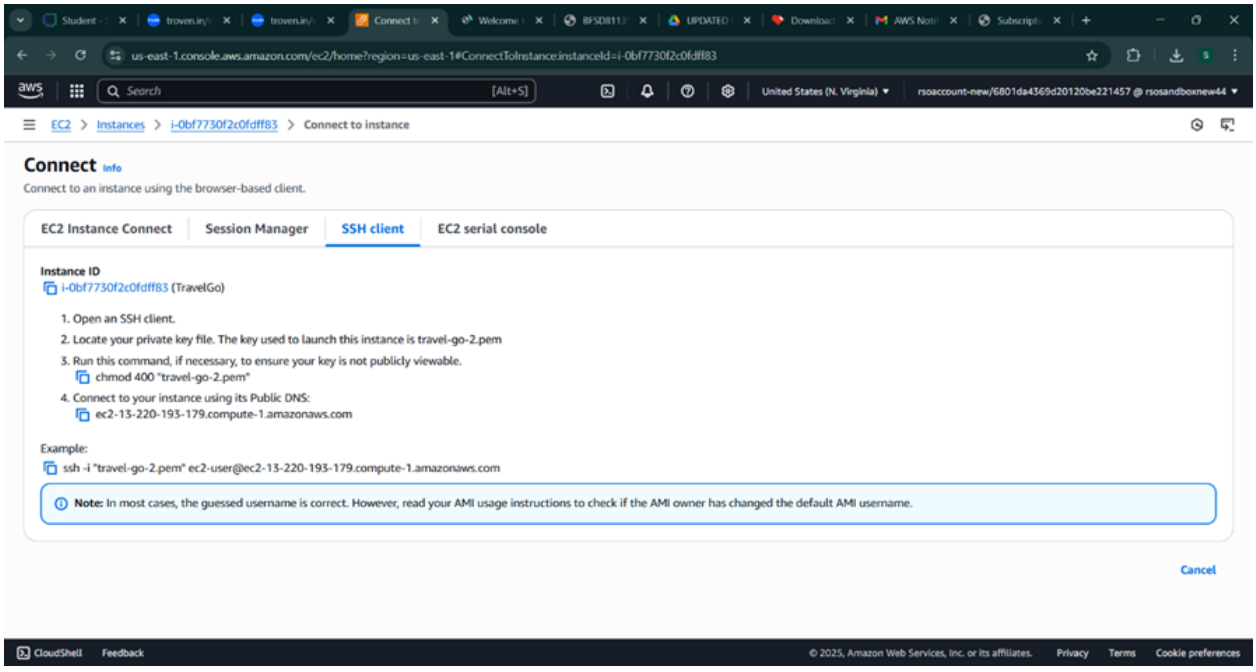


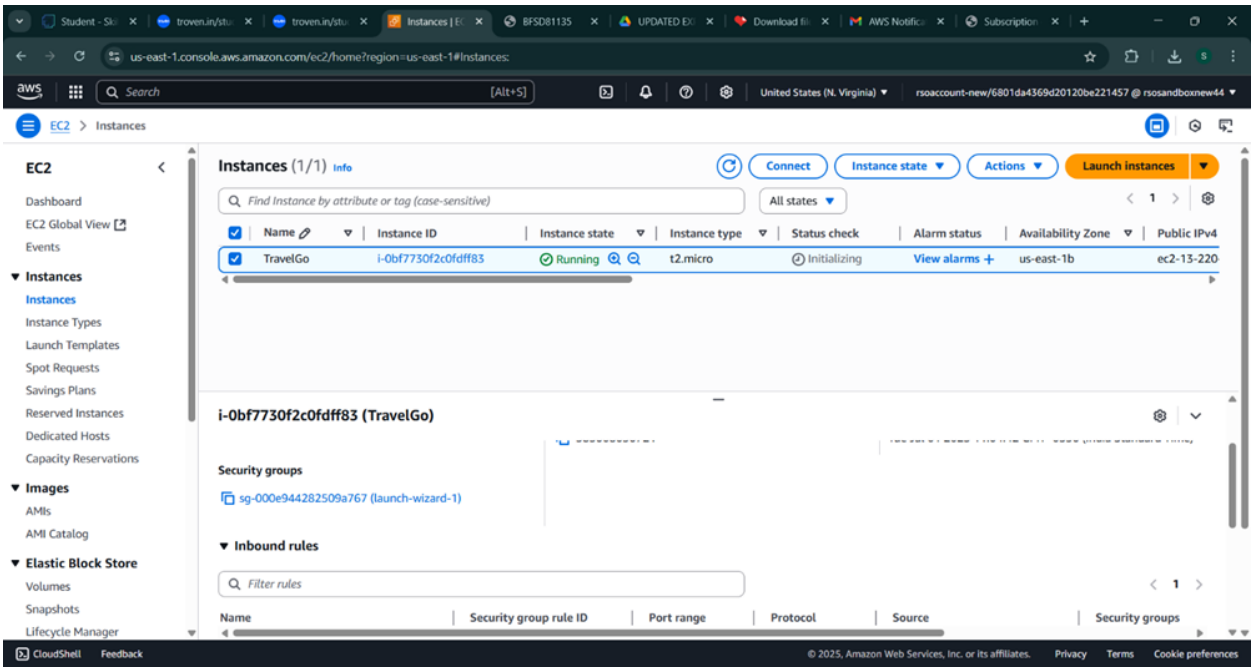
- To connect to EC2 using **EC2 Instance Connect**, start by ensuring that an **IAM role** is attached to your EC2 instance. You can do this by selecting your instance, clicking on **Actions**, then navigating to **Security** and selecting **Modify IAM Role** to attach the appropriate role. After the IAM role is connected, navigate to the **EC2** section in the **AWS Management Console**. Select the **EC2 instance** you wish to connect to. At the top of the **EC2 Dashboard**, click the **Connect** button. From the connection methods presented, choose **EC2 Instance Connect**. Finally, click **Connect** again, and a new browser-based terminalwill open, allowing you to access your EC2 instance directly from your browser.





- Now connect the EC2 with the files





Milestone 7: Deployment on EC2

Activity 7.1: Install Softwareon the EC2 Instance

Install These :

1. python3
2. Flask
3. Git:

On Amazon Linux2:

```
sudo yum update-y
sudo yum install python3 git
sudo yum install flask boto3
```

Verify Installations:

```
flask --version
git --version
```

Activity 7.2:Clone Your Flask Projectfrom GitHub

Clone your projectrepository from GitHubinto the EC2 instance using Git.

Run: 'git clone https://github.com/vikram0675/TravelGo.git'

- This will download your project to the EC2 instance.

To navigateto the project directory, run the following command:

cd TravelGo

Once inside the projectdirectory, configure and run the Flask application by executing the following command with elevated privileges:

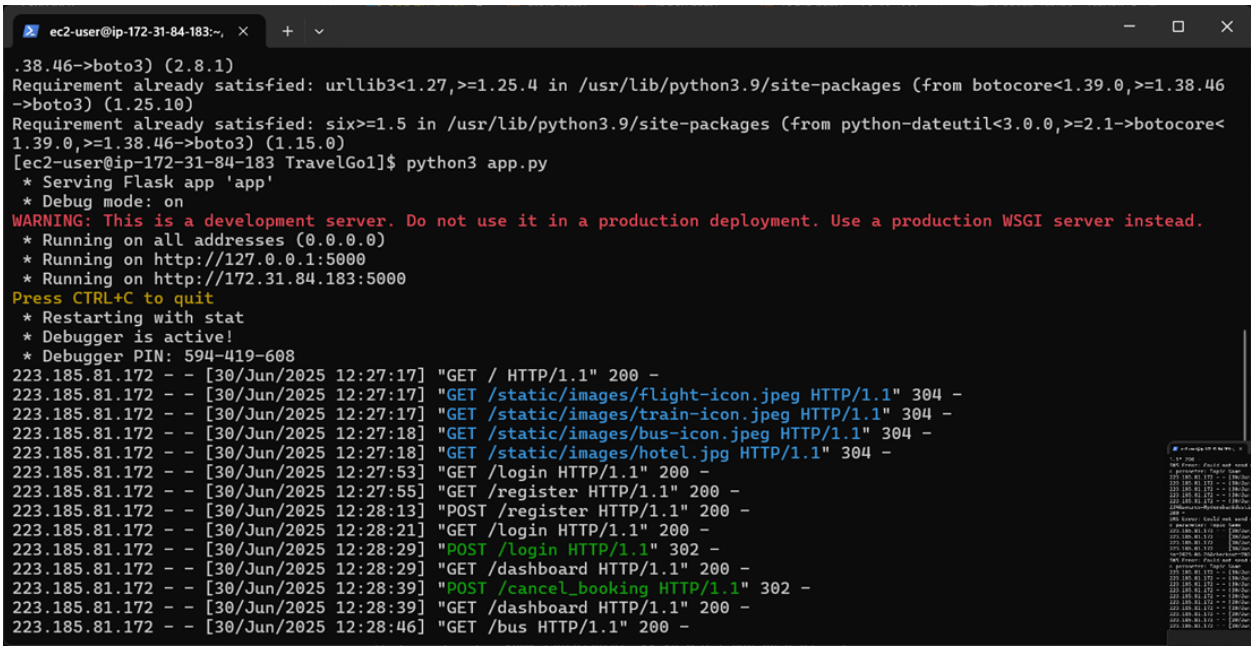
Run the Flask Application

sudo flask run --host=0.0.0.0 --port=80

Verify the Flask app is running :

http://18.205.190.145:5000/

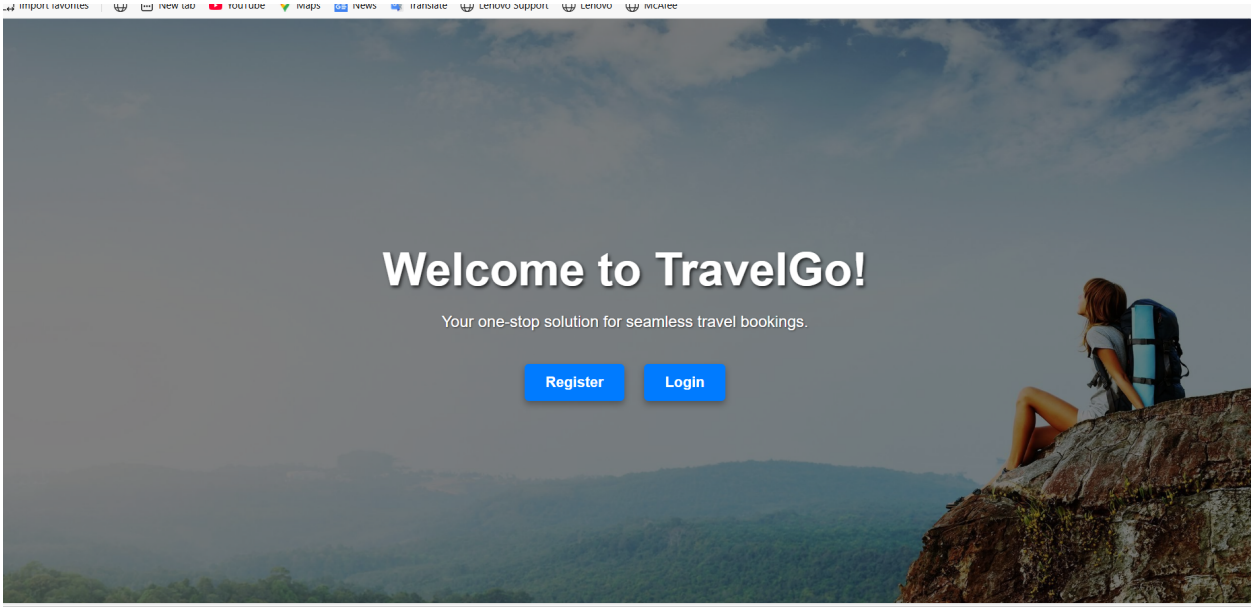
- Run the Flask app on the EC2 instance



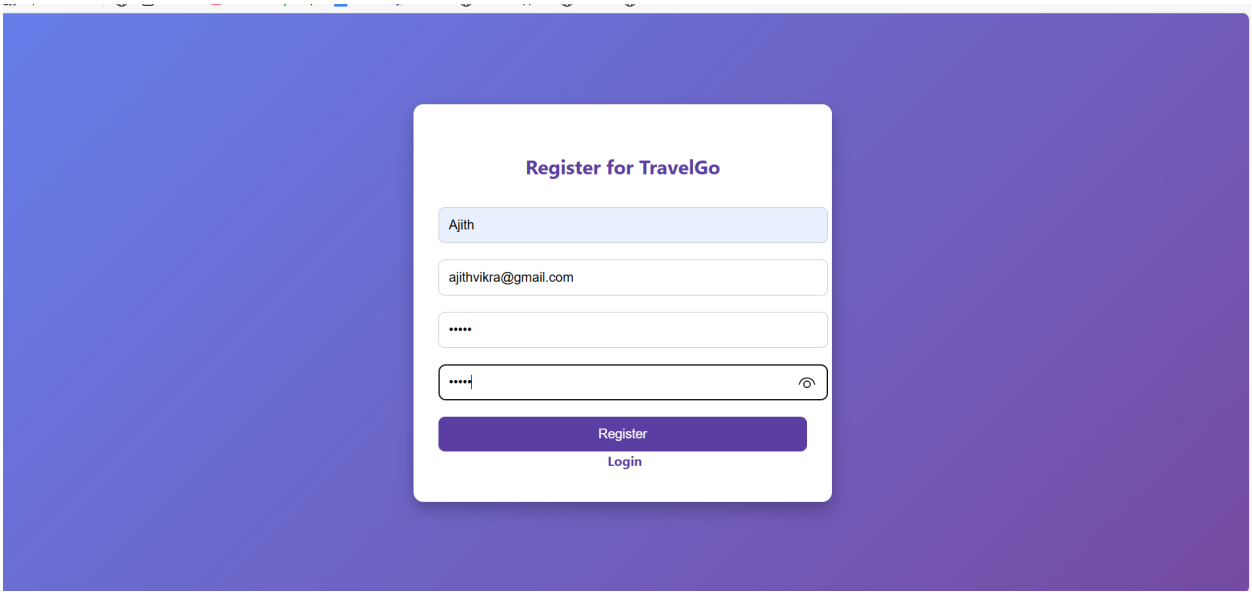
Milestone 8: Testing and Deployment

- **Activity 8.1: Conductfunctional testing to verify user registration, login,book requests, and notifications.**

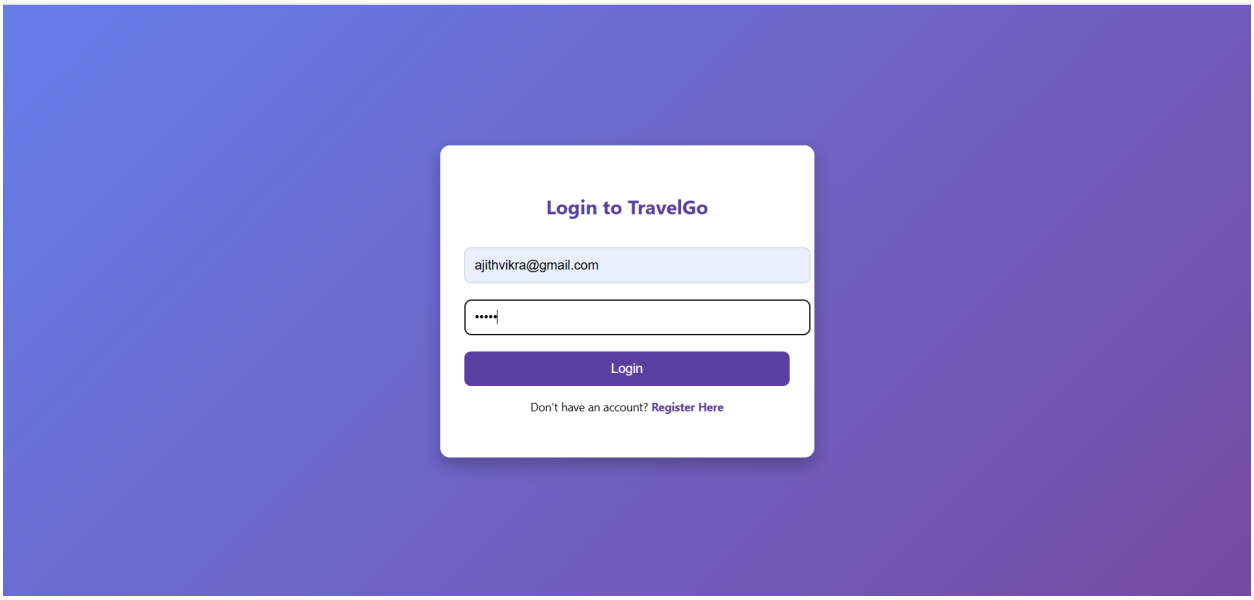
Welcome Page:



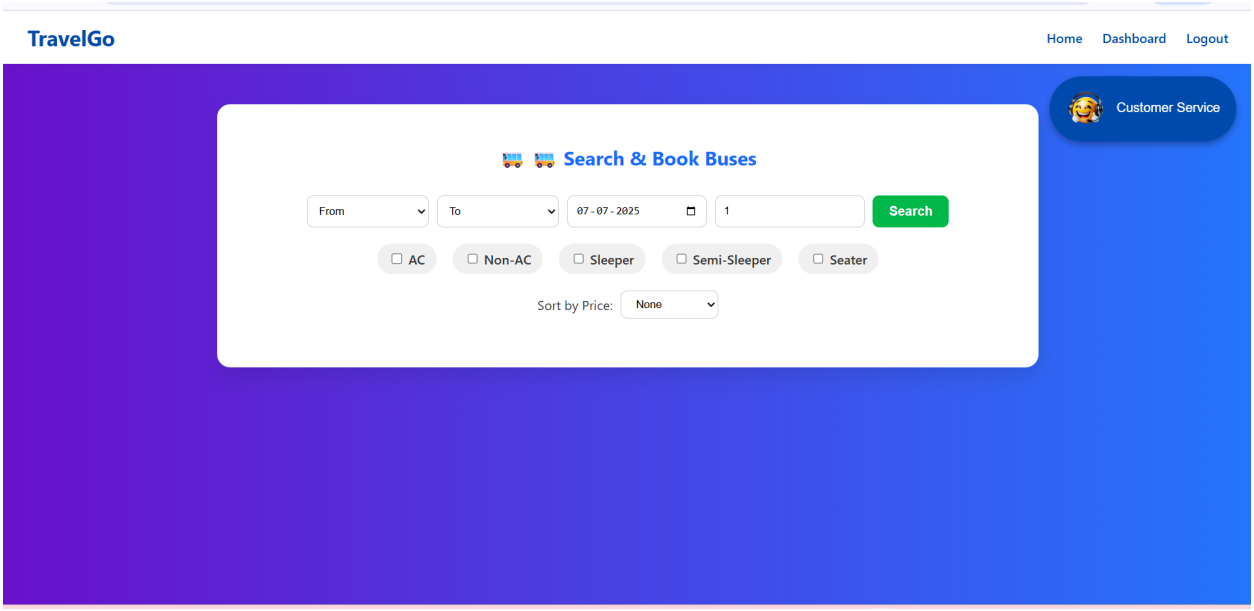
Register Page:

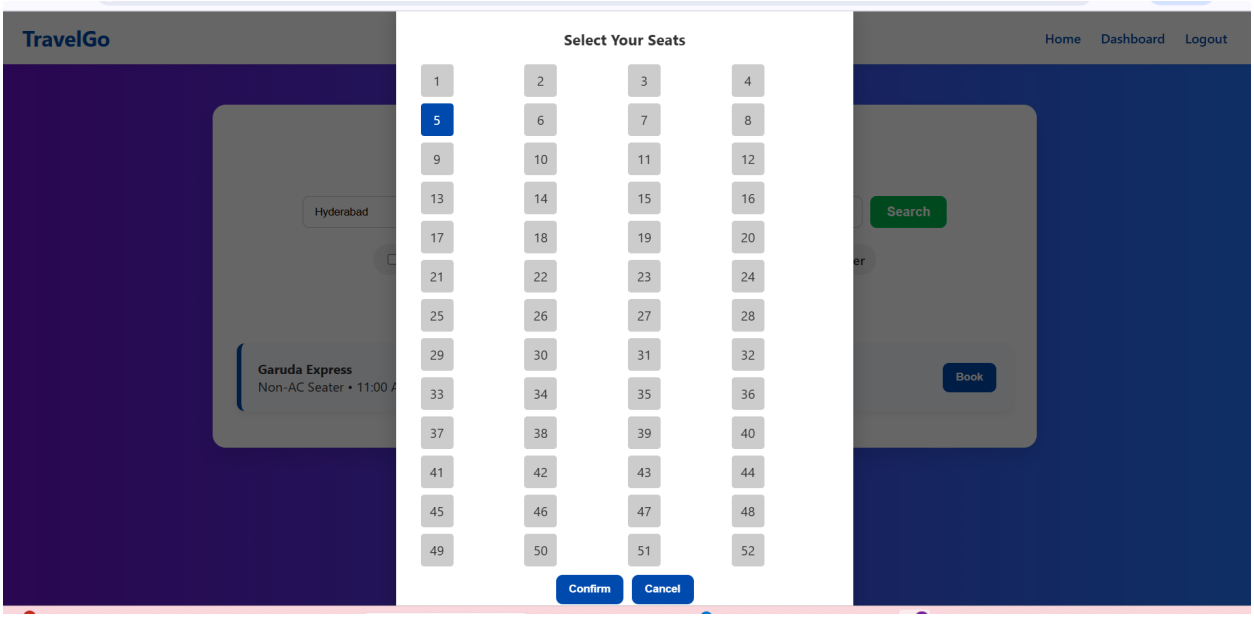


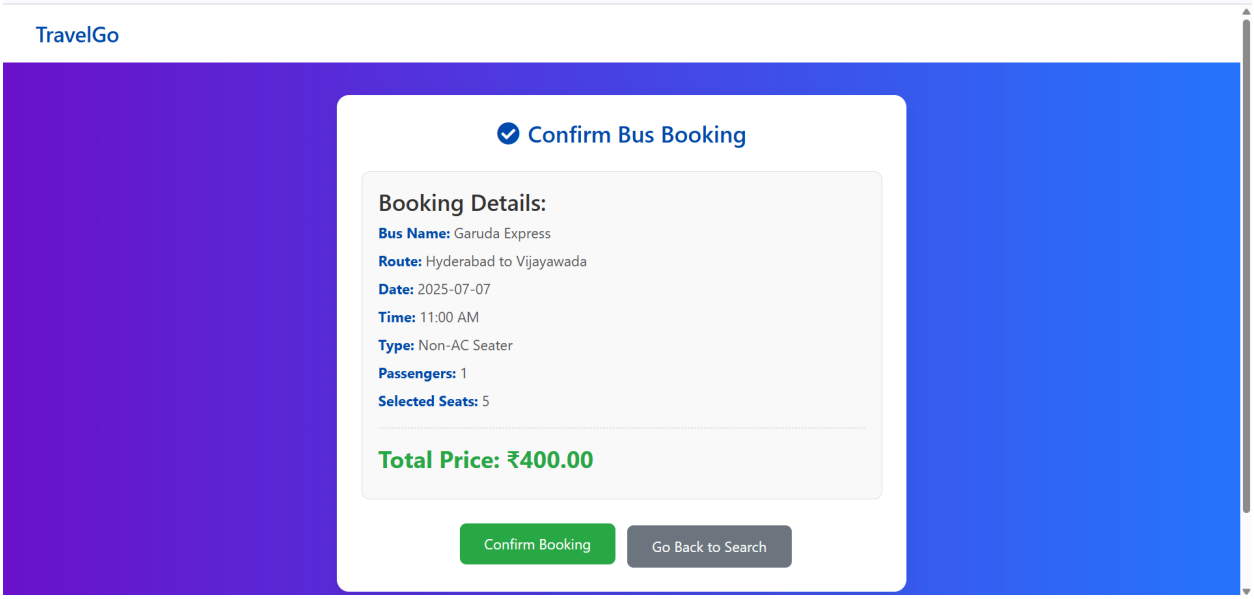
Login Page:



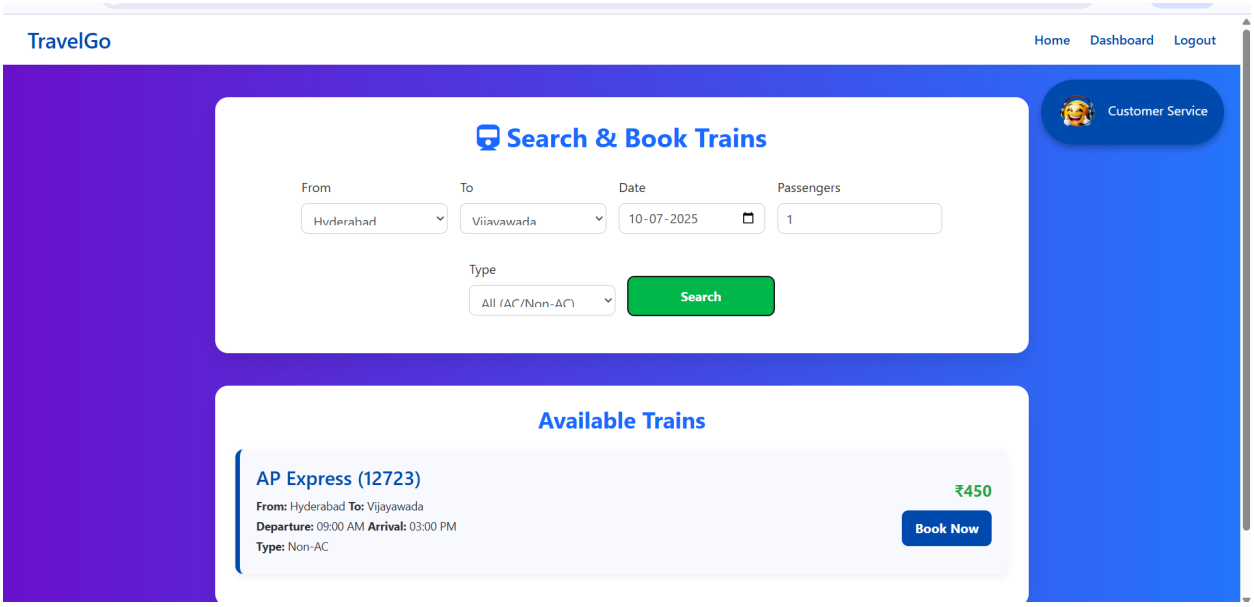
Bus Booking Page:

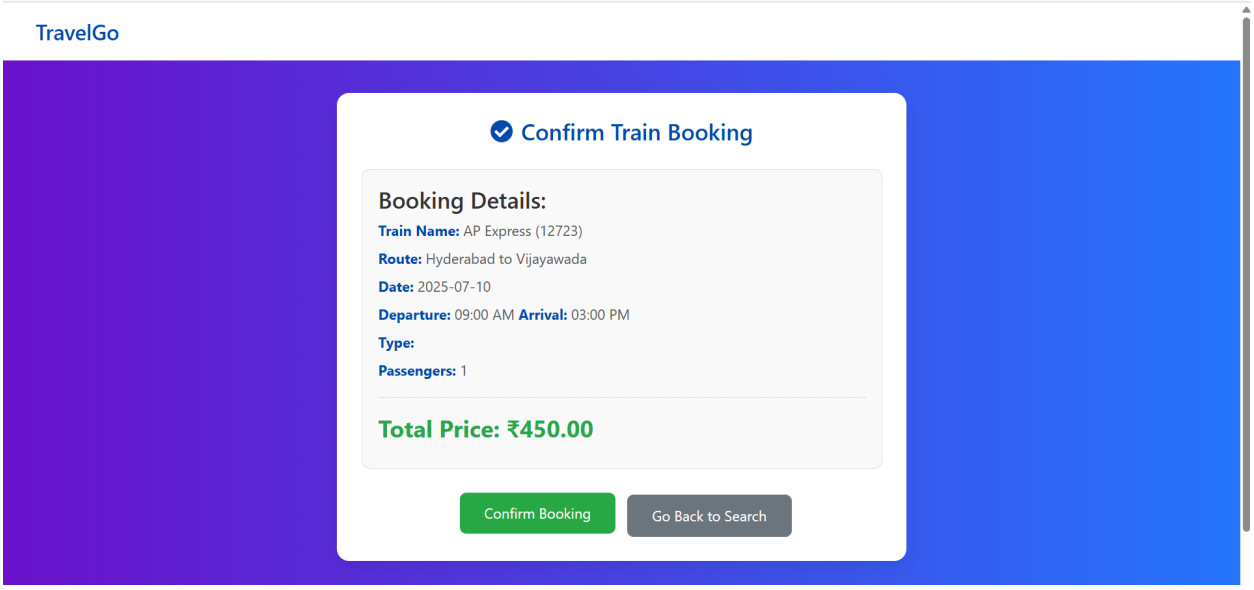



TravelGo
Home
Dashboard
Logout
Select Your Seats
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
Confirm
Cancel
Hyderabad
Search
Garuda Express
Non-AC Seater • 11:00
Book

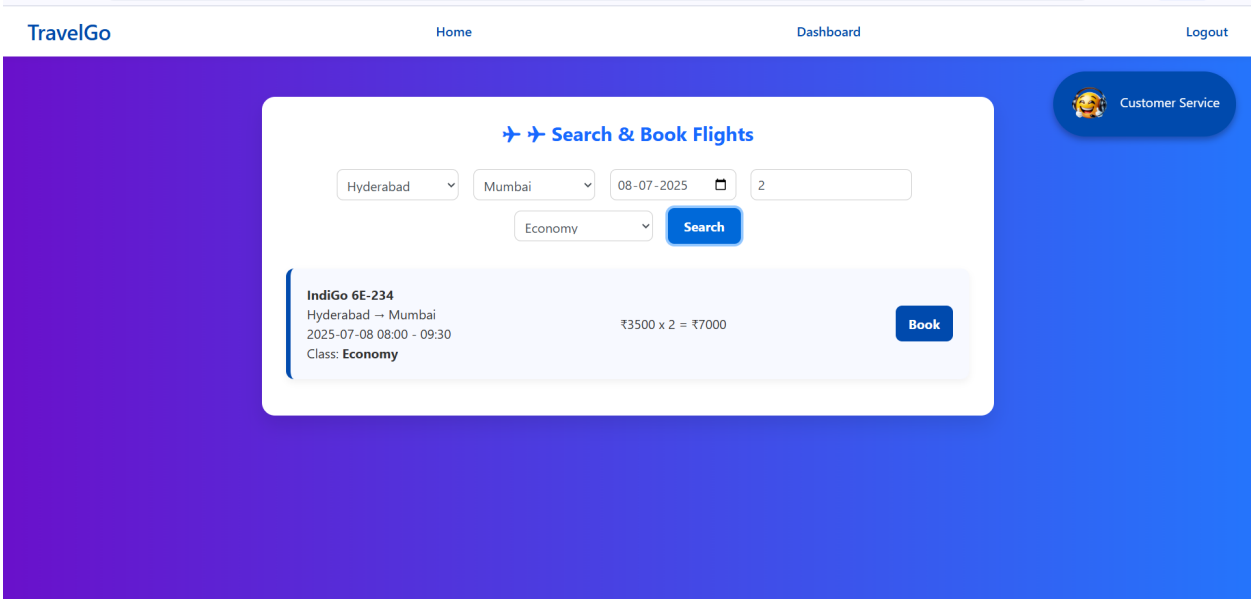

TravelGo
Confirm Bus Booking
Booking Details:
Bus Name: Garuda Express
Route: Hyderabad to Vijayawada
Date: 2025-07-07
Time: 11:00 AM
Type: Non-AC Seater
Passengers: 1
Selected Seats: 5
Total Price: ₹400.00
Confirm Booking
Go Back to Search

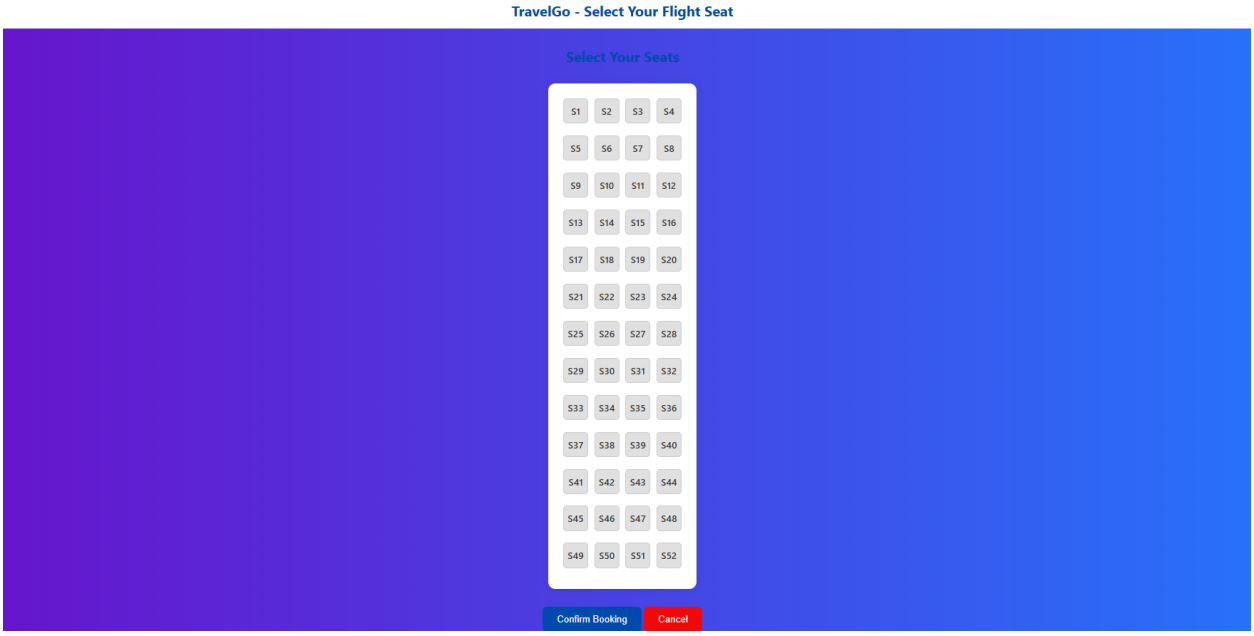
Train Booking page:



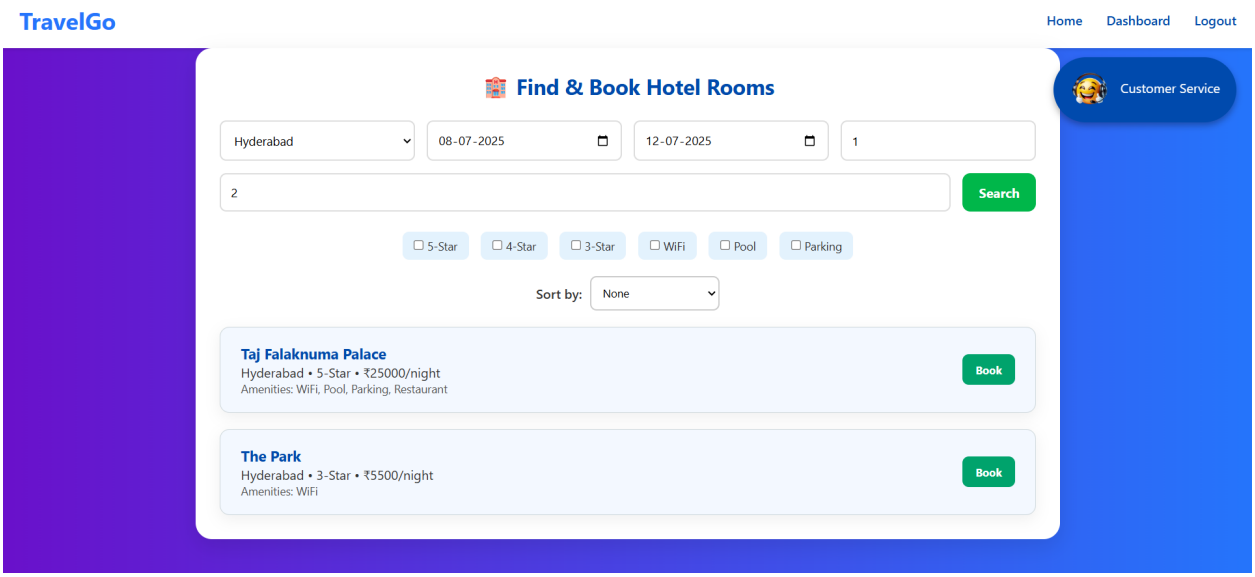


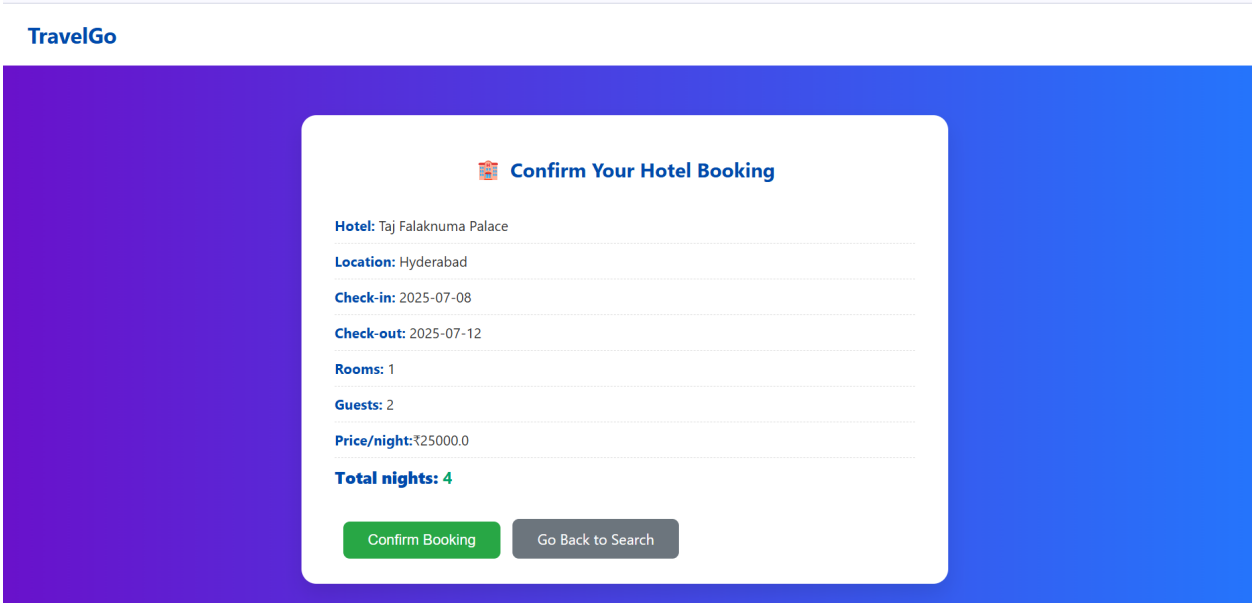
Flight Booking Page:



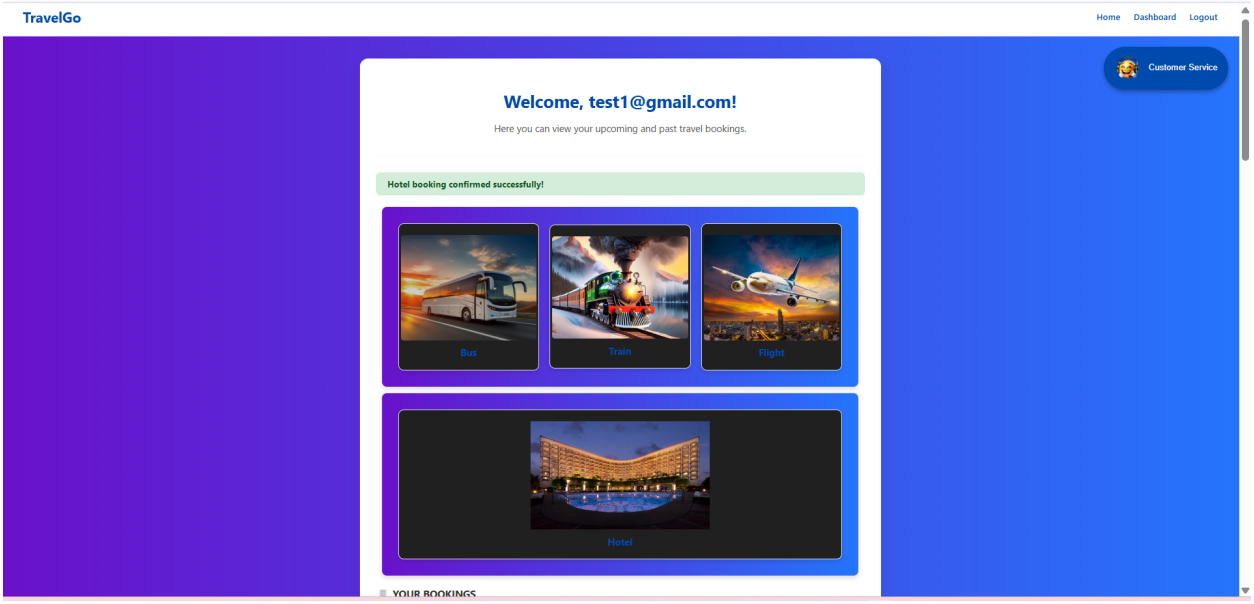


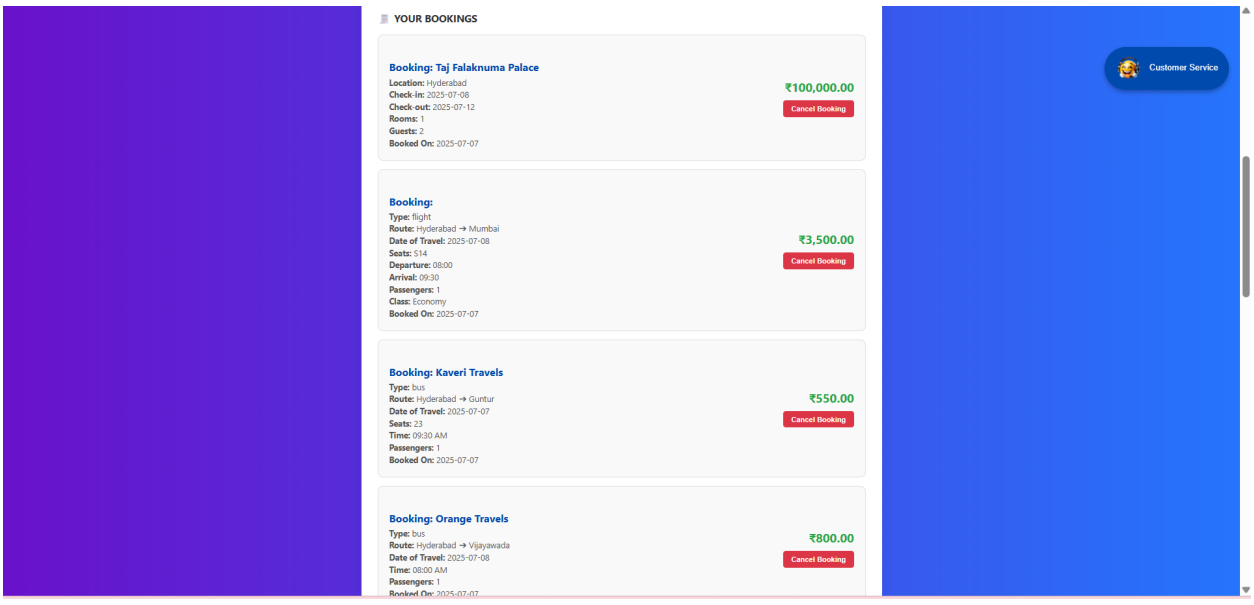
Hotel Booking Page:



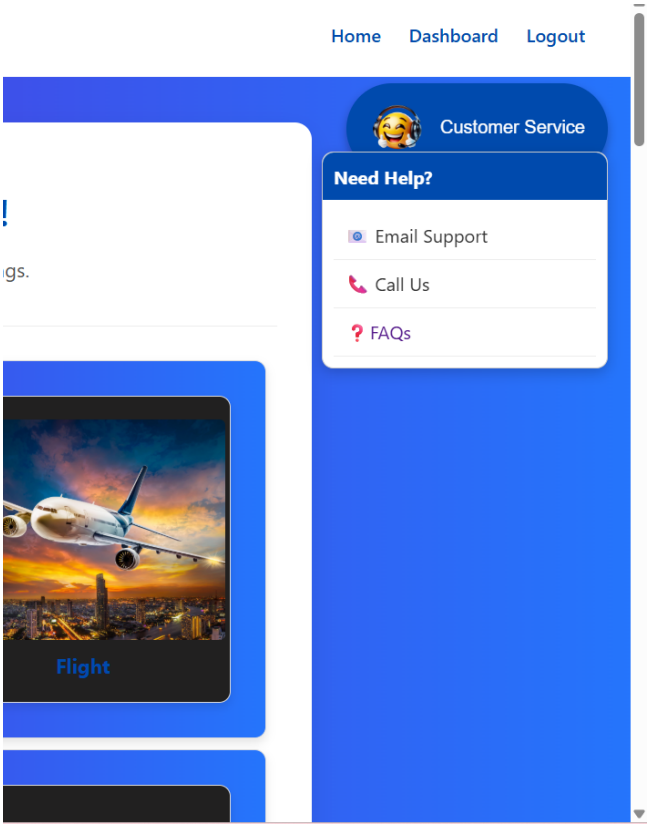


Dashboard:

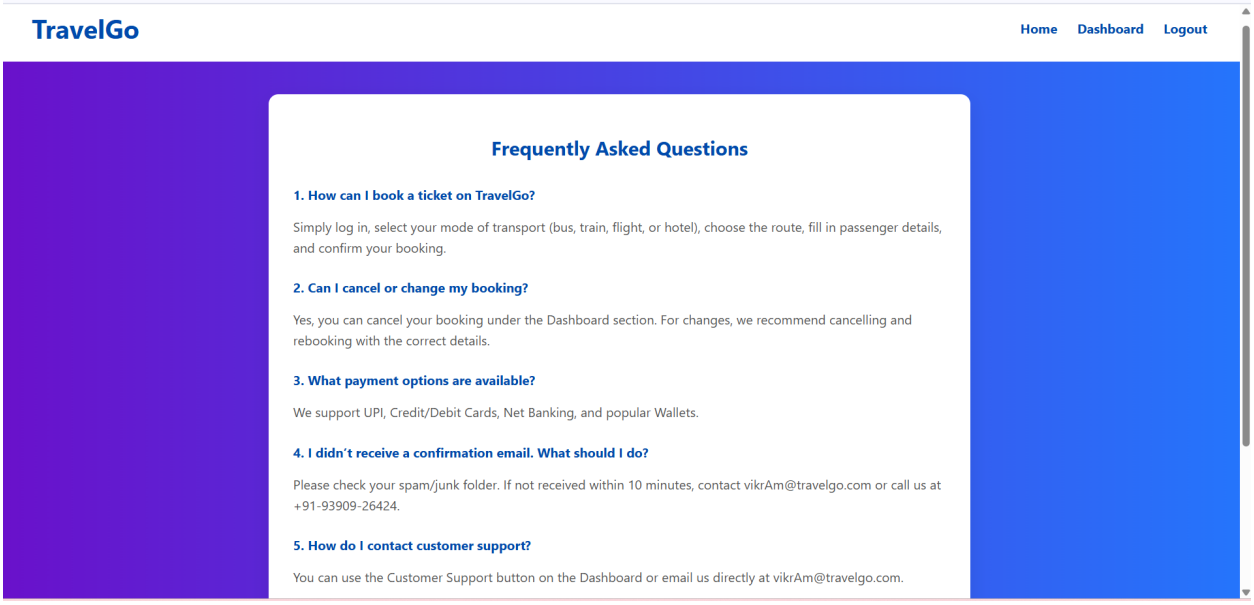

TravelGo
Home
Dashboard
Logout
Customer Service
Welcome, test1@gmail.com!
Here you can view your upcoming and past travel bookings.
Hotel booking confirmed successfully!
Bus
Train
Flight
Hotel
YOUR BOOKINGS


YOUR BOOKINGS
Booking: Taj Falaknuma Palace
Location: Hyderabad
Check-in: 2025-07-08
Check-out: 2025-07-12
Rooms: 1
Guests: 2
Booked On: 2025-07-07
₹100,000.00
Cancel Booking
Booking:
Type: flight
Route: Hyderabad → Mumbai
Date of Travel: 2025-07-08
Seats: S14
Departure: 08:00
Arrival: 09:30
Passengers: 1
Class: Economy
Booked On: 2025-07-07
₹3,500.00
Cancel Booking
Booking: Kaveri Travels
Type: bus
Route: Hyderabad → Guntur
Date of Travel: 2025-07-07
Seats: 23
Time: 09:30 AM
Passengers: 1
Booked On: 2025-07-07
₹550.00
Cancel Booking
Booking: Orange Travels
Type: bus
Route: Hyderabad → Vijayawada
Date of Travel: 2025-07-08
Time: 08:00 AM
Passengers: 1
₹800.00
Cancel Booking
Customer Service

Customer Service:



Frequently Asked Questions:



Conclusion:

The **TravelGo** Website has been successfully developed and deployed using a scalable and cloud-native architecture. Leveraging AWS services such as EC2 for hosting, DynamoDB for real-time data management, and SNS for instant booking and cancellation notifications, the platform provides a seamless travel booking experience for users. TravelGo enables registered users to search and book buses, trains, flights, and hotels in a centralized, intuitive interface, eliminating the complexities of navigating multiple travel services.

The cloud infrastructure ensures high availability and smooth performance even during peak usage, while the Flask backend ensures efficient handling of user authentication, dynamic booking flows, and data transactions. Real-time notification integration via AWS SNS allows users to receive booking confirmations and cancellations immediately via email, improving communication and user engagement.

In summary, the **TravelGo** Website offers a modern, reliable, and user-friendly solution for managing travel and accommodation needs. It highlights the potential of cloud-based platforms in building unified travel systems, simplifying operations, and enhancing the overall user experience.

COMMENTS

Document Name : **SI-27245-1751640420**

C1 ”|” in Page 1

Guest #3 *on Monday, Jul 07, 2025, 07:08 AM*

Pusuluri Ajith KHIT CSE 228x1a0583@khitguntur.ac.in